

Universidad Nacional del Nordeste

Facultad de Ciencias Exactas y Naturales y Agrimensura

Licenciatura en Sistemas de Información

PROYECTO FINAL DE CARRERA

Argos: plataforma inteligente de AIOps para la detección, resolución asistida y documentación automática de incidentes en entornos web

Autor: Juan Agustín Acosta (LU 57851)

Profesor Orientador: Rios, Leopoldo José

Profesor Coordinador:

Tribunal Examinador:

Año de Presentación: 2026

Resumen

Los equipos que operan servicios web en producción reciben alertas que indican qué métrica superó un umbral, pero no por qué ocurrió ni cómo se corrige. La respuesta depende de quién esté de guardia y de lo que recuerde, lo que alarga el tiempo medio de resolución (MTTR) y hace que problemas ya resueltos se vuelvan a investigar desde cero.

Este Proyecto Final de Carrera presenta Argos, una plataforma de AIOps construida sobre un stack de observabilidad de código abierto (Prometheus, Loki, Grafana, Alertmanager y Grafana Alloy) desplegado con Docker Compose y Terraform. Cuando las alertas superan un umbral, un flujo orquestado en n8n recupera contexto de una base de conocimiento vectorial (Qdrant, con embeddings locales) mediante recuperación aumentada de información (RAG), consulta un modelo de lenguaje a través de una cadena de proveedores con respaldo y envía por correo una síntesis del problema con pasos de resolución. Si el incidente no estaba documentado, el modelo redacta la documentación técnica y la plataforma la incorpora a la base, de modo que la siguiente ocurrencia se reconoce sin volver a documentarla. Una API en .NET 10 y un panel web en React centralizan los incidentes, la base de conocimiento, las métricas de Nginx, las pruebas de carga y la configuración.

La validación combinó escenarios de tráfico con k6 (normal, pico, ataque e inundación HTTP) y la inyección de siete fallas reales. El tráfico normal no generó falsos positivos y el escenario de ataque disparó cuatro alertas que recorrieron la cadena completa. En diez corridas automáticas, Argos entregó la propuesta en 1,1 minutos desde el inicio de la alerta y en ocho de ellas la propuesta contenía el comando exacto que resolvía la falla. Una alerta nueva que la primera vez obtuvo una similitud de 0,40 se reconoció la segunda vez con 0,97 contra el documento generado por la propia plataforma. No se midió una línea base manual de MTTR, por lo que no se informa un porcentaje de reducción; esa medición queda declarada como limitación y como trabajo futuro, junto con el despliegue en la nube.

Palabras clave: AIOps, observabilidad, recuperación aumentada de información, modelos de lenguaje, gestión de incidentes.

Índice

Contenido

Resumen 1
Índice 2
Índice de figuras 3
Índice de tablas 4
Capítulo 1. Introducción 5
1.1 Breve estado del arte 6
1.2 Objetivos 8
1.3 Fundamentación 9
Capítulo 2. Metodología 11
2.1 Requisitos de la plataforma 12
2.2 Ejecución de los sprints 13
2.3 Gestión de riesgos 17
2.4 Modelo de datos 18
Capítulo 3. Herramientas y/o lenguajes de programación 20
3.1 Herramientas de seguimiento de proyecto 20
3.2 Herramientas de desarrollo 20
3.3 Declaración del uso de herramientas de IA en el PFC 22
Capítulo 4. Resultados 24
4.1 Aspectos de ética profesional vinculados con el proyecto 24
4.2 La Ley de datos personales en relación al PFC 25
4.3 Temas disciplinares incluidos en el PFC 26
4.4 Descripción de la solución tecnológica 27
4.5 Validación de la solución tecnológica 43
Capítulo 5. Conclusiones y futuros trabajos 52
Referencias 55
Anexos 57
Anexo 1. Puesta en marcha del entorno 57
Anexo 2. Variables de entorno 57
Anexo 3. Ejemplo de runbook manual 58
Anexo 4. Ejemplo de análisis generado 58
Anexo 5. Ejemplo de documento autogenerado 59
Anexo 6. Rúbrica de evaluación de las propuestas 60
Anexo 7. Material de evaluación 60

Índice de figuras

Figura 1: Sprints del Scrum adaptado y su encadenamiento 12
Figura 2: Modelo de datos de Argos (colecciones de Qdrant) 18
Figura 3: Arquitectura de la plataforma Argos 27
Figura 4: Tablero de logs de Nginx en Grafana con un registro JSON 29
Figura 5: Reglas de alerta cargadas en Prometheus 30
Figura 6: Workflow Alert Analysis en n8n 32
Figura 7: Ciclo de vida de un incidente en Argos 34
Figura 8: Capas de la Argos API 35
Figura 9: Vista principal del panel de Argos 37
Figura 10: Vista de incidentes 38
Figura 11: Detalle de un incidente NginxCriticalErrorRate con logs de Loki y análisis 39
Figura 12: Documento autogenerado en la base de conocimiento 40
Figura 13: Señales doradas de Nginx en la vista de métricas 41
Figura 14: Vista de pruebas de carga con el código del escenario normal 42
Figura 15: Configuración: modelos de IA, umbral y destinatarios 43
Figura 16: Puntaje medio de la rúbrica antes y después de la corrección del prompt 47
Figura 17: Línea de tiempo de una corrida típica 49
Figura 18: Composición del MTTR siguiendo la propuesta de Argos 49

Índice de tablas

Tabla 1: Capacidades por enfoque de monitoreo 8
Tabla 2: Requisitos funcionales 12
Tabla 3: Requisitos no funcionales 13
Tabla 4: Sprints ejecutados, entregables y validación 13
Tabla 5: Problemas técnicos encontrados y su solución 17
Tabla 6: Riesgos identificados y respuesta 17
Tabla 7: Campos principales de un incidente 19
Tabla 8: Herramientas y lenguajes por capa 20
Tabla 9: Imágenes y versiones desplegadas 21
Tabla 10: Composición de los stacks de Docker Compose 28
Tabla 11: Reglas de detección 29
Tabla 12: Rutas de Alertmanager 31
Tabla 13: Nodos del workflow Alert Analysis 32
Tabla 14: Endpoints de la Argos API por área 35
Tabla 15: Perfiles de usuario 36
Tabla 16: Vistas del panel de Argos 37
Tabla 17: Resultados de los escenarios k6 44
Tabla 18: Catálogo de fallas de la evaluación 44
Tabla 19: Clasificación del RAG con el umbral original (0,63) 45
Tabla 20: Barrido del umbral de similitud 45
Tabla 21: Disponibilidad y latencia de los modelos (corridas 1 y 2) 46
Tabla 22: Rúbrica de calidad antes y después de la corrección 47
Tabla 23: Efecto de la corrección según el contexto disponible 47
Tabla 24: Prueba de aprendizaje con dos ocurrencias de la misma alerta 48
Tabla 25: Corridas automáticas (minutos, promedio de dos corridas por falla) 49
Tabla 26: Hallazgos de la suite E2E del panel 50
Tabla 27: Cumplimiento de los objetivos específicos 51
Tabla 28: Variables de entorno principales 57
Tabla 29: Rúbrica de calidad 60
Tabla 30: Scripts de evaluación 60

Capítulo 1. Introducción

Los entornos de producción actuales combinan arquitecturas distribuidas, contenedores y exposición permanente a Internet, y eso aumenta la necesidad de visibilidad sobre el comportamiento de los servicios [1]. El monitoreo tradicional, basado en umbrales fijos sobre métricas de infraestructura, no alcanza frente al volumen de datos que generan estos sistemas [2], [3] y produce fatiga de alertas que entorpece la respuesta a incidentes [4].

En las áreas de TI y DevOps, un incidente suele analizarse sin información que explique qué servicio falló o cuál fue la causa raíz. La resolución depende entonces del conocimiento de cada integrante del equipo: el MTTR crece y aumenta el riesgo de repetir errores que ya se habían resuelto. Una alerta típica informa, por ejemplo, que el porcentaje de respuestas 5xx superó el 20 %. Quien la recibe todavía tiene que averiguar si cayó la aplicación, la base de datos o el proxy, qué comandos revelan la causa y cómo se resolvió la última vez que pasó algo parecido.

Argos es una plataforma de Inteligencia Artificial para Operaciones de TI (AIOps) que se monta sobre un stack de observabilidad de código abierto y le agrega una capa de análisis, resolución asistida y documentación automática. La observabilidad, entendida como la capacidad de inferir el estado interno de un sistema a partir de sus métricas, logs y trazas [1], es la base sobre la que opera. La validación se hizo en un entorno simulado cuyo componente central es un proxy inverso Nginx, porque sus logs HTTP describen con detalle lo que le pasa al servicio.

El informe se organiza en cinco capítulos. Este primero presenta el estado del arte, los objetivos y la fundamentación. El Capítulo 2 describe la metodología, los requisitos y la ejecución de cada sprint. El Capítulo 3 detalla las herramientas utilizadas y declara el uso de inteligencia artificial durante el trabajo. El Capítulo 4 reúne los resultados: aspectos éticos y legales, temas disciplinares, la descripción completa de la solución y su validación. El Capítulo 5 cierra con las conclusiones y las líneas de trabajo futuro. Los anexos incluyen la puesta en marcha, la configuración, ejemplos reales de salidas de la plataforma y el material de evaluación.

1.1 Breve estado del arte

Del monitoreo por umbrales a la observabilidad

Durante años los equipos de operaciones usaron herramientas como Nagios y Zabbix, que alertan con umbrales fijos sobre CPU, memoria, disco y red. Funcionan bien cuando la infraestructura es estable y conocida: un conjunto de servidores con nombre propio, cada uno con sus chequeos. Con la adopción de microservicios y contenedores, esa premisa dejó de cumplirse. Los servicios aparecen y desaparecen, se replican y se mueven entre nodos, y la pregunta relevante pasó de "¿está vivo el servidor?" a "¿el servicio responde bien a sus usuarios?".

Prometheus y Grafana se consolidaron como estándar de facto para esa nueva situación. Prometheus recolecta métricas como series temporales identificadas por etiquetas, las consulta con un lenguaje propio (PromQL) y evalúa reglas de alerta sobre ellas; Grafana las visualiza en tableros [5]. La gestión de logs evolucionó hacia sistemas centralizados como Loki, que indexa solo las etiquetas de cada flujo y no el texto completo de cada línea, lo que reduce el costo de almacenamiento y permite consultar con LogQL, un lenguaje similar a PromQL [6]. Alertmanager completa el circuito: agrupa, deduplica, silencia y enruta las alertas hacia correo, webhooks u otros receptores [7].

La práctica de Site Reliability Engineering propuso cuatro señales doradas para monitorear un servicio expuesto a usuarios: latencia, tráfico, errores y saturación [8]. En [2] se sistematizan los patrones de diseño de trazas y métricas para aplicaciones nativas de la nube, y en [3] se releva cómo se usan esos datos para detectar anomalías, desde umbrales estáticos hasta modelos estadísticos y de aprendizaje automático.

Gestión de incidentes y MTTR

La gestión de incidentes se mide con indicadores de tiempo. El tiempo medio de detección (MTTD) va desde que empieza la falla hasta que se emite la alerta. El tiempo medio de resolución (MTTR) va desde el inicio del incidente hasta que el servicio vuelve a la normalidad, e incluye el diagnóstico, la reparación y la verificación. En [9] se muestra que la mayor parte del MTTR suele estar en el diagnóstico: entender qué pasó y qué hacer. Ese es el tramo donde una

herramienta de apoyo puede aportar más, porque la detección ya la resuelven las reglas y la reparación muchas veces es un comando conocido.

Las organizaciones intentan acortar el diagnóstico con runbooks: documentos que describen, para cada tipo de alerta, los síntomas, las causas habituales y los pasos de resolución. Su problema es el mantenimiento. Se escriben después de un incidente grave, quedan desactualizados y no cubren los casos nuevos, que son justamente los que más tiempo llevan.

AIOps y modelos de lenguaje

El término AIOps agrupa las técnicas que aplican inteligencia artificial a las operaciones de TI: correlación y reducción de ruido de alertas, detección de anomalías, análisis de causa raíz y automatización de la respuesta [9]. En [1] se propone AlertGuardian, un sistema que gestiona el ciclo de vida completo de las alertas en nubes de gran escala, y en [9] se ofrece una revisión de la literatura con pautas técnicas para soluciones de gestión de incidentes.

Los modelos de lenguaje de gran tamaño (LLM) abrieron otra línea de trabajo. Entienden el contexto semántico de un log, detectan patrones que una regla no expresa y explican en lenguaje natural lo que encuentran. LogEval [4] propone un banco de pruebas para medir su desempeño en análisis de logs, y en [10] se reporta un caso de soporte de software a gran escala. Su principal riesgo es la alucinación: el modelo puede inventar comandos, servicios o archivos que no existen en el entorno, con la misma seguridad con la que describe los que sí existen.

Recuperación aumentada de información y búsqueda vectorial

La recuperación aumentada de información (RAG) [11] mitiga ese riesgo dándole al modelo, en el momento de la consulta, documentos propios de la organización recuperados por similitud semántica. No hace falta reentrenar el modelo: basta con mantener actualizada la base de documentos. Para recuperar por significado y no por palabras exactas, cada texto se convierte en un vector (embedding) mediante un modelo de codificación de oraciones. En [12] se entrenaron modelos multilingües de este tipo por destilación, entre ellos `paraphrase-multilingual-MiniLM-L12-v2`, que produce vectores de 384 dimensiones y admite español.

La cercanía entre dos textos se mide con la similitud del coseno, el producto escalar de los vectores dividido por el producto de sus normas: vale 1 cuando apuntan en la misma dirección y se acerca a 0 cuando no tienen relación. Las bases de datos vectoriales, como Qdrant [13], indexan millones de vectores con estructuras de grafos navegables (HNSW) [14] para devolver los más cercanos en milisegundos. El umbral a partir del cual dos textos se consideran "el mismo caso" depende del modelo y del dominio, y debe calibrarse con datos propios, como se hizo en este proyecto.

Brecha que aborda Argos

La Tabla 1 resume las capacidades de los distintos enfoques. La mayoría de las soluciones de AIOps se concentra en el diagnóstico y en reducir el ruido de alertas. Son pocas las que cierran el ciclo generando y guardando documentación técnica reutilizable a partir de cada incidente analizado. Argos apunta a esa brecha: su diferencial es la autodocumentación de los incidentes que no estaban registrados, de modo que la base de conocimiento crece con el uso.

Tabla 1: Capacidades por enfoque de monitoreo

Capacidad	Monitoreo clásico (umbrales)	Stack de observabilidad	AIOps con LLM	Argos
Alertas por umbral	Sí	Sí	Sí	Sí
Logs centralizados y consultables	No	Sí	Sí	Sí
Diagnóstico en lenguaje natural	No	No	Sí	Sí
Contexto propio de la organización (RAG)	No	No	Parcial	Sí
Documentación automática de casos nuevos	No	No	Escasa	Sí
Reconocimiento de casos ya documentados	No	No	Parcial	Sí

Fuente: elaboración propia.

1.2 Objetivos

Objetivo general. Diseñar e implementar una plataforma de software que detecte incidentes a partir de umbrales de alertas, genere mediante LLM y RAG una síntesis del problema con pasos de resolución, y produzca en forma automatizada documentación técnica de los incidentes no documentados.

Objetivos específicos.

1. Implementar la arquitectura base de observabilidad (proxy inverso, API REST y base de datos) como superficie HTTP, y un nodo centralizado con Prometheus, Loki, Grafana y Alertmanager, desplegado como infraestructura como código con Terraform sobre LocalStack y Docker Compose, con migración posterior a un servicio en la nube, y definir las reglas de detección por umbrales que actúan como disparador del flujo inteligente.
2. Implementar la capa inteligente de Argos: una base de conocimiento vectorial con su pipeline RAG y un motor de orquestación que, ante un disparador, recupere contexto, consulte un LLM para generar una síntesis del problema con pasos de resolución y active la autodocumentación de los incidentes no registrados.
3. Validar el comportamiento de la plataforma con escenarios simulados con k6 (tráfico normal, picos de carga y patrones de ataque), evaluando la detección, la calidad de la propuesta generada por el LLM y la correcta persistencia de la documentación.

1.3 Fundamentación

Resolver incidentes en producción es uno de los mayores costos operativos de TI: cada minuto de indisponibilidad afecta ingresos, reputación y confianza de los usuarios. Cuando no hay monitoreo adecuado ni documentación ordenada, encontrar la causa raíz depende del conocimiento tácito de pocas personas, lo que alarga el MTTR y expone a la organización a repetir errores ya resueltos.

Desde la disciplina, el proyecto integra temas que la carrera trata por separado: redes y protocolos (tráfico HTTP, proxy inverso, limitación de tasa), sistemas operativos y virtualización (contenedores, recursos, exporters), ingeniería de software (arquitectura en capas, pruebas automatizadas, gestión ágil) e inteligencia artificial aplicada (embeddings, búsqueda vectorial, modelos de lenguaje). El aporte al ámbito de implementación es una plataforma que automatiza la captura, el análisis y la documentación de incidentes para depender menos de expertos puntuales y convertir cada incidente en conocimiento reutilizable [1], [9]. Al reunir detección [3], diagnóstico asistido por LLM [4], [10] y documentación automática en un único flujo, ofrece una

alternativa a las herramientas de monitoreo tradicionales [2], que no aprovechan el conocimiento generado en cada incidente.

Alcance

El proyecto se limita a un entorno local reproducible: un único host con Docker que ejecuta todos los componentes, incluida una aplicación de ejemplo (una API de membresías con SQL Server) que hace de servicio monitoreado. El tráfico y las fallas son simulados. Todo el código, las reglas, los flujos y los scripts de evaluación están versionados, de modo que la plataforma puede levantarse con un solo comando y los resultados de este informe pueden repetirse.

Quedan fuera del alcance la ejecución automática de acciones correctivas (Argos propone, una persona decide y ejecuta), la detección por contenido de los requests (por ejemplo, firmas de inyección SQL, que requerirían un WAF) y el despliegue en múltiples nodos.

Capítulo 2. Metodología

Se adoptó un marco ágil basado en Scrum adaptado (Scrum técnico) [15]. Los proyectos de infraestructura y observabilidad avanzan de forma incremental: cada capa necesita que la anterior funcione antes de poder probarse. El trabajo se organizó según tres principios:

- Entregas incrementales de arquitectura: cada sprint entrega un componente técnico que habilita el siguiente (slicing horizontal).
- Timeboxing: sprints de dos a tres semanas para acotar el análisis y llegar a algo ejecutable.
- Definición de terminado verificable: cada sprint cierra con entregables concretos (contenedores desplegados, reglas que disparan, flujos que corren de punta a punta) y con una validación registrada.

Adaptación de Scrum a un proyecto individual

En [15] se muestra que la mayoría de las organizaciones adapta Scrum a su contexto en lugar de aplicarlo tal cual. En este proyecto la adaptación principal fue de roles: el alumno ocupó a la vez los roles de dueño del producto y de equipo de desarrollo, y el profesor orientador actuó como interesado principal, revisando los avances y orientando las prioridades. Las ceremonias se simplificaron. La planificación de cada sprint consistió en tomar los ítems del backlog que habilitaban el siguiente incremento; la revisión fue la validación de punta a punta registrada al cierre; la retrospectiva quedó plasmada en los problemas encontrados y sus soluciones, que se documentaron como "gotchas" para no repetirlos.

Los artefactos fueron el product backlog (`docs/PRODUCT_BACKLOG.md`), organizado por sprint con el estado de cada ítem (pendiente, en progreso, hecho), la deuda técnica transversal y las dependencias entre sprints; la definición de terminado de cada sprint; y el registro de cambios del repositorio, que conserva qué se hizo,

cuándo y con qué evidencia. La Figura 1 muestra los seis sprints planificados en la propuesta.

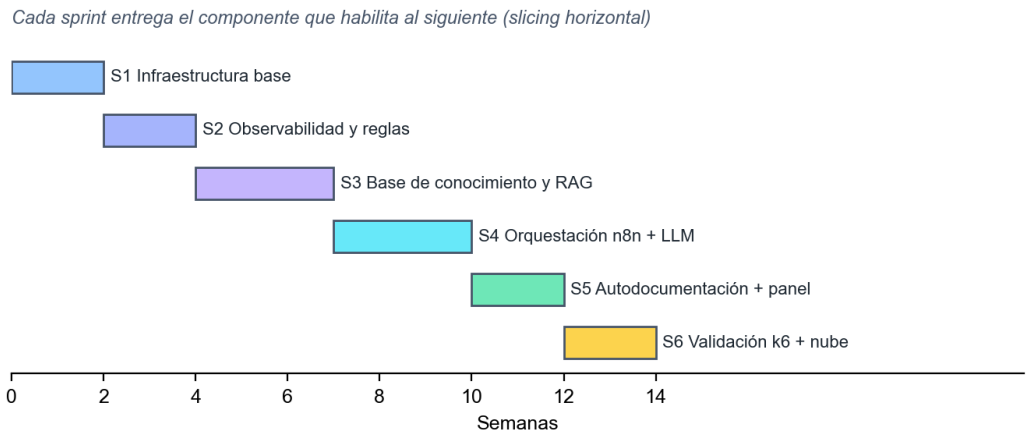


Figura 1: Sprints del Scrum adaptado y su encadenamiento

Fuente: elaboración propia.

2.1 Requisitos de la plataforma

A partir de los objetivos se definieron los requisitos funcionales (Tabla 2) y no funcionales (Tabla 3). Los requisitos guiaron el backlog y se usaron como criterio de aceptación en la validación de cada sprint.

Tabla 2: Requisitos funcionales

ID	Requisito
RF-01	Recolectar métricas del host, de los contenedores y de Nginx, y los logs de acceso y de error de Nginx.
RF-02	Evaluar reglas de alerta por umbral sobre métricas y logs y enrutar las alertas según su tipo y severidad.
RF-03	Recibir las alertas en un flujo orquestado y filtrar las que no justifican un análisis (umbral de cantidad o severidad crítica).
RF-04	Buscar en la base de conocimiento los incidentes semánticamente parecidos a la alerta y clasificarla como conocida o nueva según un umbral configurable.
RF-05	Generar con un LLM una síntesis del problema y pasos de resolución, usando como contexto los incidentes recuperados.
RF-06	Notificar por correo a un conjunto configurable de destinatarios.
RF-07	Documentar automáticamente los incidentes nuevos y agregarlos a la base de conocimiento.
RF-08	Registrar cada ocurrencia de un incidente conocido con la solución del caso que coincidió.
RF-09	Registrar la resolución de cada alerta y calcular el MTTR real.
RF-10	Ofrecer un panel web con autenticación y roles para consultar incidentes, base de conocimiento, métricas y configuración.
RF-11	Permitir que el administrador cambie en caliente el proveedor y modelo de IA, el umbral de similitud, la autodocumentación y los destinatarios.
RF-12	Ejecutar escenarios de carga bajo demanda desde el panel y mostrar sus resultados.

Fuente: elaboración propia.

Tabla 3: Requisitos no funcionales

ID	Requisito
RNF-01	Disponibilidad del análisis: la caída de un proveedor de LLM no debe impedir que el incidente se registre ni que se envíe la notificación.
RNF-02	Privacidad: los embeddings se calculan localmente; los logs completos no salen del servidor.
RNF-03	Costo: la plataforma debe poder operar con niveles gratuitos de los proveedores de LLM.
RNF-04	Reproducibilidad: todo el entorno se levanta con un solo comando a partir del repositorio.
RNF-05	Seguridad: autenticación con tokens, contraseñas con hash, control de acceso por rol y endpoints internos protegidos.
RNF-06	Latencia: la propuesta debe llegar en pocos segundos una vez que Alertmanager entrega la alerta.
RNF-07	Mantenibilidad: arquitectura en capas con dependencias por interfaz y configuración por variables de entorno.
RNF-08	Idempotencia: reiniciar la plataforma o recibir alertas repetidas no debe duplicar incidentes ni documentos.

Fuente: elaboración propia.

2.2 Ejecución de los sprints

La Tabla 4 resume, para cada sprint, el entregable y la validación con la que se dio por cerrado. A continuación se describe cada uno con sus decisiones y los problemas que aparecieron.

Tabla 4: Sprints ejecutados, entregables y validación

Sprint	Entregable	Validación / cierre
1. Infraestructura base	Terraform sobre LocalStack (EC2 simulado) y tres stacks Docker Compose (monitoring, proxy, apps) en la red `argos-net`; API de membresía en .NET con SQL Server inicializado por `db-init`.	Flujo Nginx → Alloy → Loki → Grafana funcionando (agosto de 2026).
2. Observabilidad y reglas	Prometheus con 13 reglas, Loki con 5 reglas LogQL, Alertmanager con ruteo y tres tableros de Grafana.	Reglas visibles y evaluadas en Prometheus y en el ruler de Loki.
3. Base de conocimiento y RAG	Qdrant con la colección `incidentes`, embeddings locales de 384 dimensiones y carga de runbooks.	Consultas de similitud con umbral inicial 0,63 (6 de agosto de 2026).
4. Orquestación n8n + LLM	Workflow Alert Analysis: webhook, umbral de alertas, RAG, LLM y correo.	Prueba E2E con alerta simulada y correo enviado (11 de agosto de 2026).
5. Autodocumentación y panel	Rama de autodocumentación, API completa de incidentes y panel propio con siete vistas.	Validación E2E (23 de septiembre de 2026).
6. Validación y nube	Escenarios k6 (también on demand desde el panel), evaluación del OE3 y suite E2E con Playwright.	k6 validado (30 de septiembre); evaluación cerrada (28 de octubre). Despliegue en la nube pendiente.

Fuente: elaboración propia.

Sprint 1. Infraestructura base

Se configuró el provider de AWS de Terraform contra el endpoint de LocalStack y se definió una instancia EC2 simulada (`t3.medium`, etiquetada `argos-node`). Un script de PowerShell registra o recupera una AMI de Ubuntu 22.04 en LocalStack, y un recurso `null_resource` orquesta el levantamiento de los tres stacks de Docker Compose en orden: monitoreo, aplicaciones y proxy. Los stacks se comunican por una red externa compartida, `argos-net`, creada una sola vez. El stack de aplicaciones incluye una API mínima en .NET 10 y SQL Server 2022; un contenedor `db-init` ejecuta un script idempotente que crea el esquema, el procedimiento almacenado y los datos iniciales en cada arranque.

Más adelante el despliegue se desacopló de Terraform: un `docker-compose.yml` en la raíz incluye los tres stacks y levanta toda la plataforma con `docker compose up -d --build`. Terraform conserva la definición de la instancia y el provisionamiento opcional (`provision_docker=true`), que es la base para la migración a la nube.

Sprint 2. Observabilidad y reglas de detección

Prometheus quedó configurado con cuatro trabajos de recolección (el propio Prometheus, node-exporter, cAdvisor y nginx-exporter) cada 15 segundos. Loki usa almacenamiento en disco con esquema TSDB, retención de siete días y un ruler que evalúa las reglas LogQL y envía las alertas a Alertmanager. Grafana Alloy, sucesor de Promtail, lee los archivos de log de Nginx desde un volumen compartido: el access log en formato JSON, del que extrae 16 campos, y el error log, que procesa con expresiones regulares. Grafana tiene provisionados los orígenes de datos y tres tableros de solo lectura: resumen de Nginx, infraestructura y logs de Nginx.

En este sprint se definieron las 18 reglas de detección descritas en la sección 4.4. Las cinco reglas sobre los logs de Nginx se pensaron como el disparador central de Argos, porque describen la experiencia del usuario del servicio (errores, latencia y tráfico anómalo) y no solo el estado de la infraestructura.

Sprint 3. Base de conocimiento y RAG

Se agregó Qdrant al stack de monitoreo y se creó la colección `incidentes` con vectores de 384 dimensiones y distancia coseno. El modelo de embeddings se

eligió local para no depender de una API externa ni pagar por consulta, y para que los textos de los incidentes no salgan del servidor. Se cargaron cinco runbooks escritos a mano, en Markdown con un encabezado YAML (tipo de alerta, resumen, pasos y estado): errores 5xx, latencia de Nginx, IP sospechosa, picos en el error log y recursos del host. La consulta devuelve los tres incidentes más parecidos con su puntaje y una clasificación: documentado si supera el umbral, nuevo si no.

El umbral inicial de 0,63 surgió de pruebas manuales. Este modelo multilingüe produce puntajes más bajos que otros modelos en inglés, del orden de 0,5 a 0,65 para textos relacionados, por lo que los valores habituales de 0,8 no servían. En el Sprint 6 el umbral se recalibró con datos de la evaluación.

Sprint 4. Orquestación con n8n y LLM

Se incorporó n8n con almacenamiento SQLite y se agregó a Alertmanager el receptor `webhook-n8n`. El primer workflow recibía la alerta, aplicaba el umbral de cantidad, consultaba el RAG, armaba el prompt, llamaba al LLM y enviaba el correo. Se implementó soporte para dos proveedores (Gemini y Claude) seleccionables por variable de entorno, con nodos que adaptaban la petición y la respuesta a cada formato. La validación de punta a punta con una alerta simulada terminó con el correo recibido el 11 de agosto de 2026.

Dos problemas marcaron este sprint. El chequeo de salud de n8n fallaba porque `localhost` resolvía a IPv6 dentro del contenedor y el servicio escuchaba en IPv4; se resolvió usando `127.0.0.1`. Y al reimportar el workflow se duplicaban las versiones; se fijó un identificador estable (`argos-alert-analysis`) para que cada importación reemplace a la anterior. Más tarde, en octubre, la llamada al LLM se trasladó de n8n a la Argos API, lo que permitió implementar la cadena de respaldo y quitar las claves de los proveedores del motor de orquestación.

Sprint 5. Autodocumentación y panel

Se tomó la decisión de usar Qdrant como única fuente de persistencia para Argos. Los incidentes se guardan con su vector y con un payload enriquecido (documento en Markdown, severidad, fechas de detección y resolución, MTTR y estado), de modo que la misma colección sirve como historial del panel y como

base de conocimiento. Se descartaron PostgreSQL con pgvector y SQL Server porque obligaban a sincronizar dos almacenes para el mismo dato.

En el workflow se agregó la rama de autodocumentación: si el RAG clasifica la alerta como nueva, un segundo prompt pide al modelo un documento con secciones fijas (título, descripción, causa raíz probable, pasos de resolución y prevención) y el resultado se guarda con estado `auto_documentado`. En paralelo se construyó el panel con React, Vite y TypeScript, servido por Nginx, que redirige `/api` a la Argos API para evitar problemas de CORS.

La validación detectó un error que dejaba la base de conocimiento siempre vacía en el panel: el filtro buscaba los documentos por origen (`fuente=auto-generado`) y no incluía los runbooks manuales. Se corrigió filtrando por estado (`documentado_previamente` o `auto_documentado`). También se detectó que `gemini-flash-latest` es un modelo con razonamiento interno cuyos pensamientos consumen el límite de tokens de salida: con 1 024 tokens las respuestas llegaban cortadas. Se elevó el límite a 4 096 para el análisis y 6 144 para la documentación.

Sprint 6. Validación, pruebas de carga y mejoras

Este sprint fue el más largo porque reunió la validación con k6, la evaluación del tercer objetivo y varias mejoras que surgieron de esas pruebas. En septiembre se validaron los tres escenarios originales de k6 y la cadena de detección completa. En octubre se agregó la ejecución de k6 bajo demanda desde el panel, el panel de señales doradas de Nginx, el pool de destinatarios, la configuración funcional y la migración de la API de Python a .NET 10 con arquitectura en capas.

Un incidente real cambió el diseño: una alerta `CAdvisorDown` se perdió porque `gemini-flash-latest` devolvió un error 503 por saturación y el workflow se cortó sin registrar nada. A partir de ese caso se implementó la cadena de respaldo de proveedores, el seguimiento pasivo de su salud y las notificaciones en el panel. También se agregó el ciclo de vida de las alertas (resolución y MTTR real) y se corrigió el texto con el que se guardan los incidentes, que era distinto del texto con el que se buscan y hacía que una alerta no se reconociera a sí misma (0,398 de similitud). La evaluación del OE3 cerró el 28 de octubre de 2026 y el despliegue en la nube quedó pendiente.

Problemas técnicos destacados

La Tabla 5 reúne problemas que aparecieron durante la ejecución y que no eran evidentes de antemano. Quedaron documentados para quien mantenga la plataforma.

Tabla 5: Problemas técnicos encontrados y su solución

Problema	Causa	Solución
Healthcheck de n8n siempre fallido	`localhost` resolvía a IPv6 dentro del contenedor.	Usar `127.0.0.1` en el chequeo.
Respuestas del LLM cortadas	Los tokens de razonamiento de Gemini consumían `maxOutputTokens`.	Subir el límite a 4 096 y 6 144 tokens.
k6 no podía escribir sus resultados	La imagen de k6 corre con uid 12345 y el volumen pertenece a root.	Lanzar el contenedor hermano como `root`.
Tasa de error de k6 siempre 0	En `--summary-export` las métricas de tipo Rate guardan la fracción en `value`, no en `rate`.	Leer la clave `value`.
Series de Loki vacías en el panel	`query_range` alinea las muestras a múltiplos del paso.	Alinear la grilla del backend con `(start//step)*step`.
Panel sin conexión tras recrear la API	El Nginx del panel cachea la IP del contenedor.	Reiniciar `argos-dashboard` después de recrear la API.
Salud de servicios en "desconocido"	La API, Qdrant y el RAG no tenían trabajos de Prometheus.	Sondas HTTP directas sobre `argos-net`.
El rate limit contaba como error 5xx	Nginx responde 503 por defecto al limitar.	`limit_req_status 429`.
Una alerta no se reconocía a sí misma	Se guardaba solo el resumen y se buscaba con resumen y descripción.	Guardar el mismo `query_text` con el que se busca.

Fuente: elaboración propia.

2.3 Gestión de riesgos

Durante la planificación se identificaron riesgos técnicos y se definió una respuesta para cada uno (Tabla 6). Dos de ellos se materializaron: la indisponibilidad del proveedor de LLM y la calidad variable de las respuestas.

Tabla 6: Riesgos identificados y respuesta

Riesgo	Impacto	Respuesta
Caída o saturación del proveedor de LLM	Alertas sin análisis ni registro.	Cadena de respaldo con tres proveedores; el incidente se registra aunque falle toda la cadena. Se materializó.
Costo de las APIs	Proyecto no sostenible.	Embeddings locales y niveles gratuitos de Gemini, Groq y OpenRouter.
Respuestas inventadas del modelo	Comandos inexistentes o peligrosos.	RAG con runbooks, descripción del stack en el prompt y evaluación con rúbrica. Se materializó y se mitigó.
Umbral de similitud mal calibrado	Soluciones equivocadas o documentos duplicados.	Umbral configurable en caliente y barrido con fallas de resultado conocido.
Recursos insuficientes del host	Contenedores inestables durante las pruebas.	Límites de memoria, exporters livianos y un único stack.

Riesgo	Impacto	Respuesta
Exposición de datos sensibles	Filtración de IP o credenciales.	Embeddings locales, `.env` fuera del repositorio, token interno y puertos en loopback.

Fuente: elaboración propia.

2.4 Modelo de datos

El modelo de datos se muestra en la Figura 2. Cada colección de Qdrant guarda documentos JSON (payload). La colección `incidentes` agrega el vector de 384 dimensiones que se usa en la búsqueda por similitud; las demás colecciones usan un vector mínimo de relleno porque Qdrant exige uno, y se consultan solo por sus campos.

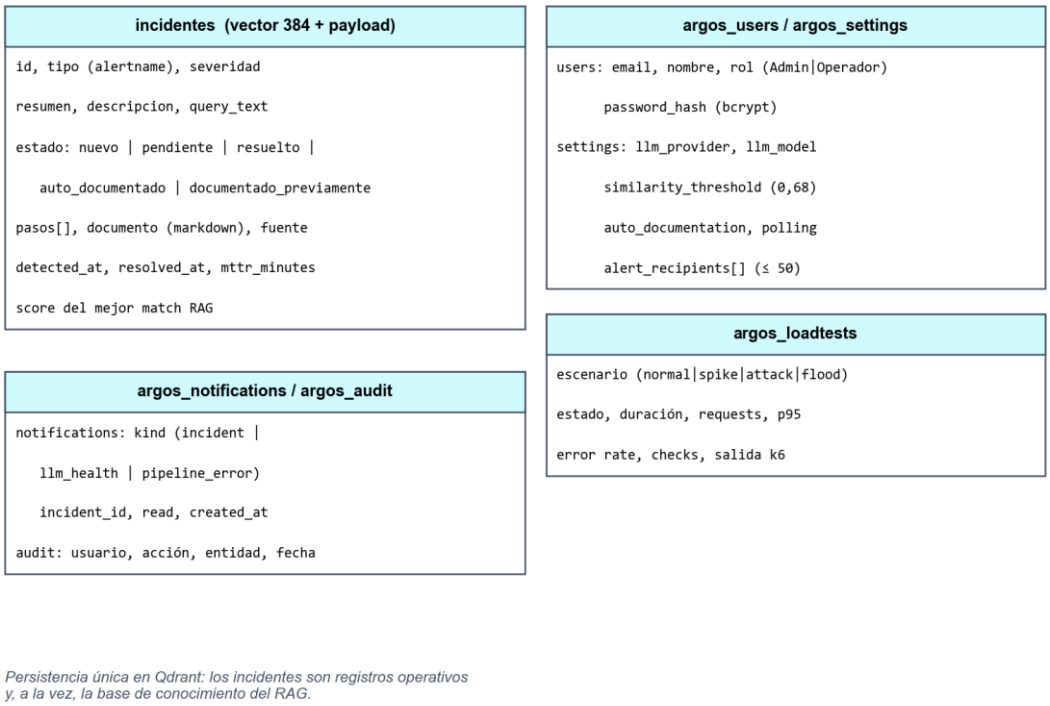


Figura 2: Modelo de datos de Argos (colecciones de Qdrant)

Fuente: elaboración propia.

Los identificadores de los incidentes son determinísticos: un UUID versión 5 calculado a partir del tipo de alerta, el resumen y la fecha de detección. Guardar dos veces el mismo incidente lo actualiza en lugar de duplicarlo, lo que hace idempotentes el arranque y las repeticiones de Alertmanager. La Tabla 7 describe los campos principales de un incidente.

Tabla 7: Campos principales de un incidente

Campo	Descripción
`tipo_alerta`	Nombre de la regla que disparó la alerta (por ejemplo, `NginxCriticalErrorRate`).
`resumen`	Texto con el que se calcula el embedding: resumen y descripción de la alerta.
`pasos`	Lista de pasos de resolución (runbooks manuales o solución copiada).
`documento`	Documentación en Markdown, generada por el LLM o escrita a mano.
`severity`	Severidad de la alerta: `warning` o `critical`.
`estado`	`nuevo`, `pendiente`, `resuelto`, `auto_documentado` o `documentado_previamente`.
`fuente`	Origen del registro: `manual`, `alertmanager` o `auto-generado`.
`detected_at` / `resolved_at`	Inicio y fin de la alerta según Alertmanager.
`mttr_minutes`	Diferencia entre la resolución y la detección, en minutos.

Fuente: elaboración propia.

Capítulo 3. Herramientas y/o lenguajes de programación

Las herramientas se agrupan según su uso: seguimiento del proyecto, desarrollo de la plataforma y asistencia con inteligencia artificial. En todos los casos se priorizaron herramientas de código abierto o con niveles gratuitos, que se pudieran ejecutar en contenedores y que tuvieran documentación pública.

3.1 Herramientas de seguimiento de proyecto

- Git y GitHub para el control de versiones de todo el proyecto: infraestructura, configuración, código, flujos de n8n y scripts de evaluación.
- Product backlog en Markdown (`docs/PRODUCT_BACKLOG.md`), organizado por sprint, con el estado de cada ítem, la deuda técnica y las dependencias entre sprints.
- Registro de cambios en `CLAUDE.md` y documentos de cierre por tema (`k6/RESULTS.md`, `docs/EVALUACION_OE3.md`, `docs/FIXES_DASHBOARD.md`, `docs/FUNCIONALIDADES_DASHBOARD.md`), que conservan la evidencia de cada validación.

Mantener el backlog y los registros dentro del repositorio tuvo una ventaja práctica: cada cambio de código queda junto a la decisión que lo motivó, y cualquier persona que clone el proyecto encuentra el estado y la historia sin depender de una herramienta externa.

3.2 Herramientas de desarrollo

La Tabla 8 enumera las herramientas y lenguajes utilizados, agrupados por capa, y la Tabla 9 las versiones de las imágenes de contenedor desplegadas.

Tabla 8: Herramientas y lenguajes por capa

Capa	Herramientas	Uso en Argos
Infraestructura	Terraform [16], LocalStack [17], Docker y Docker Compose [18]	Infraestructura como código, simulación de AWS y despliegue de los tres stacks con un compose raíz.
Superficie HTTP	Nginx, ASP.NET Core (.NET 10) [19], SQL Server 2022	Proxy inverso con limitación de tasa, API de membresía de ejemplo y su base de datos.
Observabilidad	Prometheus [5], Loki [6], Alertmanager [7], Grafana, Grafana Alloy [20], exporters	Métricas, logs, 18 reglas de alerta, ruteo y tableros.

Capa	Herramientas	Uso en Argos
Inteligencia	Qdrant [13], ONNX Runtime [21], modelo paraphrase-multilingual-MiniLM-L12-v2 [12]	Base vectorial y embeddings locales de 384 dimensiones.
Modelos de lenguaje	Google Gemini [22], Groq [23], OpenRouter [24]; Anthropic Claude como opción	Síntesis, pasos de resolución y documentación, con cadena de respaldo.
Orquestación	n8n [25] (SQLite)	Workflow que une alertas, RAG, LLM, correo y persistencia.
Backend de Argos	C# / .NET 10, Autofac, JWT, bcrypt, Docker.DotNet	API en capas: autenticación, incidentes, RAG, LLM, observabilidad y pruebas de carga.
Frontend	TypeScript, React [26], Vite, Tailwind CSS	Panel web de siete vistas.
Pruebas	k6 [27], Playwright [28], PowerShell y Node.js	Carga y ataque, pruebas E2E del panel y scripts de evaluación.

Fuente: elaboración propia.

Tabla 9: Imágenes y versiones desplegadas

Componente	Imagen y versión	Puerto
Prometheus	`prom/prometheus:v2.54.1`	9090
Loki	`grafana/loki:3.2.0`	3100
Grafana	`grafana/grafana:11.3.0`	3000
Alertmanager	`prom/alertmanager:v0.27.0`	9093
Grafana Alloy	`grafana/alloy:v1.4.3`	12345
node-exporter	`prom/node-exporter:v1.8.2`	9100
cAdvisor	`gcr.io/cadvisor/cadvisor:v0.49.1`	8080
nginx-exporter	`nginx/nginx-prometheus-exporter:1.1.0`	9113
Qdrant	`qdrant/qdrant:v1.12.4`	6333
n8n	`n8nio/n8n:1.70.3`	5678
SQL Server	`mcr.microsoft.com/mssql/server:2022-latest`	1433
Nginx (proxy)	Imagen propia basada en Nginx	80 / 8443
Argos API	Imagen propia (.NET 10, multi-stage)	8000 (solo loopback)
Panel Argos	Imagen propia (build de Node, servida por Nginx)	3001

Fuente: elaboración propia.

Justificación de las elecciones principales

Loki y Grafana Alloy. Loki permite escribir reglas de alerta sobre logs con el mismo modelo que Prometheus usa para métricas, y comparte el ecosistema de Grafana. Alloy se eligió en lugar de Promtail porque es su sucesor y concentra la recolección en un único agente configurable.

Qdrant. Es una base vectorial dedicada, liviana, con API REST y filtros sobre el payload. Permitió guardar en el mismo lugar los vectores y los datos operativos de los incidentes. Las alternativas evaluadas (PostgreSQL con pgvector o una base relacional aparte) agregaban un segundo almacén para el mismo dato.

Embeddings locales con ONNX. El modelo MiniLM multilingüe cuantizado pesa poco, corre en CPU en milisegundos y entiende español. Ejecutarlo dentro de la API con ONNX Runtime evitó una dependencia externa, un costo por consulta y la salida de los textos del servidor. El tokenizer SentencePiece de XLM-R se implementó con `Microsoft.ML.Tokenizers`, con truncado a 128 tokens y promedio de los vectores de salida (mean pooling).

n8n. Es un motor de flujos de código abierto con editor visual, que permite ver la ejecución de cada nodo y sus datos. Eso facilitó depurar el flujo y presentar el proceso. La lógica de negocio pesada (RAG, cadena de LLM, persistencia) se dejó en la API para que el workflow quede como orquestador.

.NET 10 para la API. La primera versión de la API se escribió en Python. La migración a .NET permitió una arquitectura en capas con tipado estático, inyección de dependencias con Autofac y un único lenguaje con la aplicación de ejemplo, que también es .NET. El costo fue reimplementar el tokenizer y el cliente de Qdrant.

k6. En [29] se comparan herramientas de pruebas de carga para microservicios. k6 se eligió porque sus escenarios se escriben en JavaScript, define umbrales de aceptación en el propio script, corre en un contenedor y exporta un resumen en JSON que la API puede leer para mostrar los resultados en el panel.

React y Vite en lugar de Next.js. La propuesta preveía Next.js. El panel resultó ser una aplicación de una sola página detrás de un login, sin necesidad de renderizado en servidor ni de posicionamiento en buscadores, por lo que se optó por React con Vite, más simple de construir y servir como archivos estáticos.

3.3 Declaración del uso de herramientas de IA en el PFC

En el proyecto la inteligencia artificial cumple dos papeles distintos, que conviene separar.

Como componente del producto, Argos usa modelos de lenguaje (Gemini, Groq y OpenRouter, con Claude como alternativa configurable) para generar las síntesis y la documentación de los incidentes, y un modelo de embeddings local para la búsqueda semántica. Estos usos forman parte del objeto de estudio y se evalúan en el Capítulo 4.

Como herramienta de trabajo del autor, se utilizó Claude Code (Anthropic, modelo Claude Opus) como asistente de programación en la terminal. Se usó para generar y refactorizar código bajo revisión del autor (entre otros, la migración de la API de Python a .NET 10 y la suite E2E con Playwright), diagnosticar fallas del entorno, redactar documentación técnica del repositorio y preparar un primer borrador de este informe a partir de esa documentación. En la evaluación del OE3, Claude actuó además como evaluador asistido que puntuó con una rúbrica las respuestas de los modelos; el autor revisó esas puntuaciones y la limitación se declara en la sección 4.5. Las decisiones de diseño, la ejecución de las pruebas, la verificación de los resultados y la redacción final son responsabilidad del autor.

Capítulo 4. Resultados

En este capítulo se exponen los resultados vinculados al diseño y desarrollo de la solución: los aspectos éticos y legales, los temas de la disciplina que integra el proyecto, la descripción de la plataforma y su validación.

4.1 Aspectos de ética profesional vinculados con el proyecto

Se tomó como referencia el Código de Ética y Conducta Profesional de la ACM [30]. El código se organiza en principios éticos generales, responsabilidades profesionales, principios de liderazgo y cumplimiento del propio código. Cuatro de sus principios se relacionan directamente con decisiones tomadas en Argos.

Principio 1.3, ser honesto y confiable. El código pide ser transparente sobre las capacidades y limitaciones de un sistema. En la evaluación no se pudo medir el tiempo de resolución de un operador humano sin la plataforma. En lugar de estimar un porcentaje de reducción del MTTR sin datos propios, el informe reporta solo lo medido (tiempo hasta la propuesta y hasta la acción correcta) y declara la ausencia de línea base como limitación. Del mismo modo, se aclara que la rúbrica de calidad la puntuó un LLM con revisión del autor y no un panel humano independiente, y que el propio informe se redactó con asistencia de IA.

Principio 1.6, respetar la privacidad. Los logs de un proxy inverso contienen direcciones IP y rutas solicitadas. Para limitar la exposición, los embeddings se calculan localmente con un modelo ONNX y los logs no salen del servidor para la búsqueda semántica. Al LLM externo solo llega el texto de la alerta y el contexto recuperado, no los logs completos.

Principio 2.5, evaluar de forma completa los sistemas y sus riesgos. Un asistente que propone comandos puede causar daño si se equivoca. La evaluación midió no solo cuándo acierta, sino también los errores más costosos: por ejemplo, un falso "conocido" que copia una solución equivocada. El umbral de similitud se ajustó a 0,68 para eliminar ese error en el catálogo de pruebas, aunque eso aumentara los documentos de más. Las propuestas se envían a una persona que decide; la plataforma no ejecuta comandos de forma autónoma.

Principio 2.9, diseñar sistemas robustos y seguros. La API exige autenticación JWT con roles (Administrador y Operador), guarda las contraseñas

con bcrypt, protege con un token interno los endpoints que usa n8n y publica su puerto solo en la interfaz local. La suite E2E detectó tres fallas de seguridad (endpoints sin autenticación y una acción reservada al administrador disponible para cualquier rol) que se corrigieron antes del cierre. La cadena de respaldo de modelos evita que la caída de un proveedor deje un incidente sin registrar.

Hay además una cuestión ética propia de las herramientas de IA en operaciones: la confianza excesiva. Un operador puede ejecutar un comando sugerido sin entenderlo. Por eso las propuestas de Argos incluyen pasos de verificación antes del arreglo, citan el runbook o documento del que provienen y avisan cuando el análisis no estuvo disponible.

4.2 La Ley de datos personales en relación al PFC

La Ley 25.326 de Protección de los Datos Personales [31] define como dato personal la información de cualquier tipo referida a personas físicas determinadas o determinables (art. 2). En Argos hay tres fuentes de este tipo de datos: las cuentas de usuario del panel (nombre, correo y rol), el pool de direcciones de correo que reciben las alertas y las direcciones IP registradas en los logs de Nginx, que pueden permitir identificar a una persona. En el caso de estudio todo el tráfico fue sintético, generado con k6, y las únicas direcciones reales son las del autor; aun así, el análisis corresponde a un despliegue en una organización real.

- **Art. 4, calidad de los datos.** Los datos deben ser adecuados y no excesivos para la finalidad. Argos guarda de cada usuario solo lo necesario para autenticarlo y asignarle un rol, y de cada incidente el texto de la alerta, no el log completo. Loki conserva los logs durante siete días; en producción ese período debería fijarse en una política escrita.
- **Art. 9, seguridad de los datos.** El responsable debe adoptar medidas técnicas que garanticen la seguridad y confidencialidad. Se aplican contraseñas con hash bcrypt, autenticación por token, control de acceso por rol, auditoría de las acciones y endpoints internos protegidos. Quedan pendientes para producción el reemplazo de las credenciales de desarrollo y certificados TLS válidos.

- **Art. 10, deber de confidencialidad.** Quienes acceden a los datos deben guardar secreto. El panel restringe la gestión de usuarios al rol Administrador y registra quién hizo cada acción.
- **Art. 12, transferencia internacional.** Enviar el texto de una alerta a un proveedor de LLM con servidores en el exterior puede constituir una transferencia de datos si ese texto incluye una IP. En el escenario F6 la descripción de la alerta contiene la IP sospechosa. Una organización que adopte Argos debería anonimizar o enmascarar las IP antes de la llamada al modelo, o verificar que el país del proveedor ofrezca un nivel de protección adecuado.

También corresponde mencionar el derecho de acceso y de rectificación (arts. 14 y 16). Los usuarios del panel pueden ver sus datos y el administrador puede corregirlos o eliminarlos desde la vista de usuarios, lo que cubre esos derechos para las cuentas. Para las IP de los logs no hay un mecanismo de consulta individual; la retención limitada es la medida que reduce el riesgo.

4.3 Temas disciplinares incluidos en el PFC

El proyecto integró y profundizó los siguientes temas de la Licenciatura en Sistemas de Información:

- Redes y comunicaciones: protocolo HTTP, códigos de estado, proxy inverso, TLS, limitación de tasa por IP (Nginx responde 429 por encima de 30 solicitudes por segundo por IP en la API) y patrones de ataque de capa 7.
- Sistemas operativos y virtualización: contenedores, redes de Docker, volúmenes, consumo de CPU y memoria medido con node-exporter y cAdvisor, y ejecución de contenedores hermanos mediante el socket de Docker.
- Ingeniería de software: requisitos, arquitectura en capas con inyección de dependencias, diseño de API REST, autenticación y autorización, pruebas automatizadas E2E y metodología ágil.
- Bases de datos: modelo relacional en SQL Server para la aplicación de ejemplo y base de datos vectorial para la búsqueda semántica.

- Inteligencia artificial: embeddings de oraciones, similitud del coseno, RAG, ingeniería de prompts y evaluación de modelos de lenguaje con rúbricas.
- Infraestructura como código y operaciones: Terraform, configuración declarativa de reglas, observabilidad (métricas, logs y alertas) y la métrica MTTR como indicador de gestión de incidentes.
- Seguridad informática: autenticación, hash de contraseñas, mínimo privilegio, encabezados de seguridad HTTP y mitigación de denegación de servicio por limitación de tasa.

4.4 Descripción de la solución tecnológica

Arquitectura general

La Figura 3 muestra los componentes de Argos y el recorrido de los datos. Todos los servicios corren en contenedores sobre la red `argos-net` y se levantan con un único `docker compose up`, que incluye tres stacks: proxy, apps y monitoring.

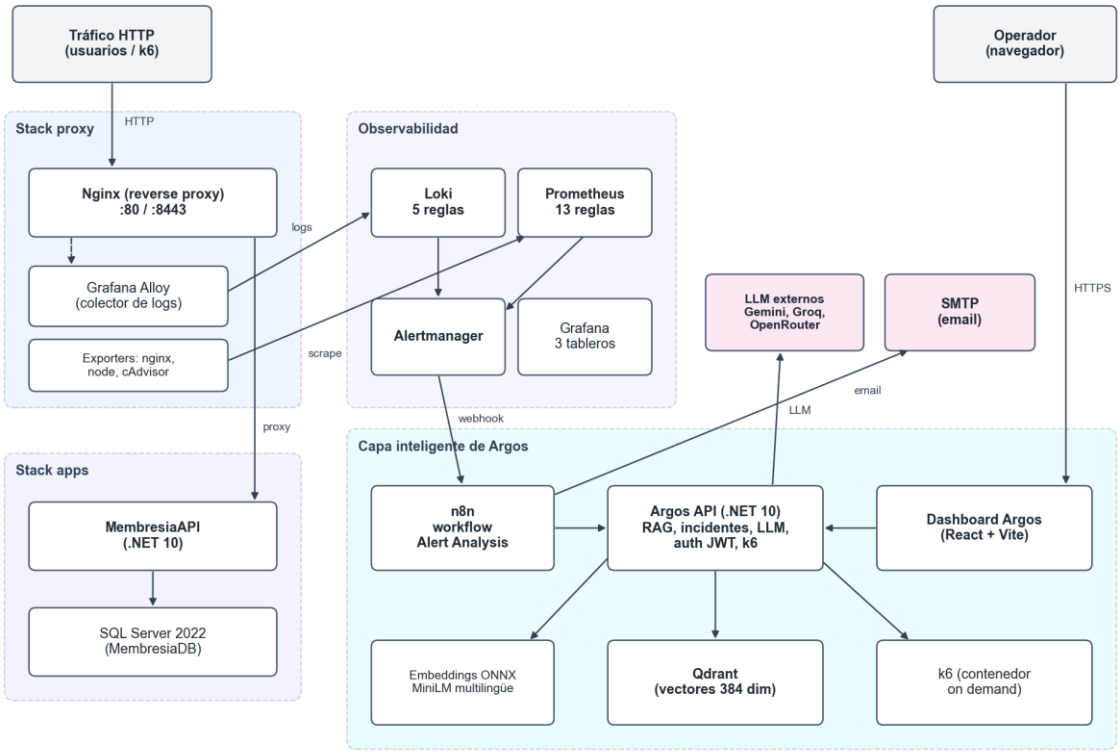


Figura 3: Arquitectura de la plataforma Argos

Fuente: elaboración propia.

El recorrido de una alerta es el siguiente. El tráfico llega a Nginx, que funciona como proxy de la API de membresía (.NET 10) y de su base SQL Server. Por un lado, Grafana Alloy lee el archivo de logs de Nginx y lo envía a Loki. Por otro, Prometheus consulta las métricas que exponen los exporters de Nginx, del host y de los contenedores. Cuando una regla se cumple, Alertmanager agrupa las alertas y las envía por webhook a n8n. El workflow le pide a la Argos API que busque incidentes parecidos en Qdrant y que genere la respuesta con el LLM. El resultado se envía por correo al pool de destinatarios y queda disponible en el panel.

La Tabla 10 detalla la composición de cada stack. La plataforma completa ejecuta 15 contenedores de larga duración, más los contenedores efímeros de inicialización de la base y de k6.

Tabla 10: Composición de los stacks de Docker Compose

Stack	Contenedores	Función
proxy	nginx-proxy, alloy, nginx-prometheus-exporter, node-exporter, cadvisor	Borde HTTP, recolección de logs y exporters de métricas.
apps	membresia-api, membresia-sqlserver, db-init (efímero)	Servicio monitoreado de ejemplo y su base de datos.
monitoring	prometheus, loki, alertmanager, grafana, qdrant, argos-n8n, argos-rag-api, argos-dashboard	Observabilidad, capa inteligente y panel.

Fuente: elaboración propia.

Superficie HTTP y recolección de logs

Nginx escribe el access log en un formato JSON propio con 19 campos, entre ellos la IP de origen, el método, la URI, el código de estado, el tiempo total del request y el tiempo de respuesta del upstream. Ese formato evita parsear líneas de texto con expresiones regulares y permite que las reglas de Loki filtren por estado o extraigan la latencia con el operador `unwrap`. Nginx aplica además encabezados de seguridad, compresión y limitación de tasa: una zona de 30 solicitudes por segundo por IP con ráfaga de 100 para la API, un máximo de 10 conexiones simultáneas por IP y respuesta 429 cuando se excede, para que el

rate limit no se confunda con un error del backend. La Figura 4 muestra el tablero de Grafana con los logs de Nginx tal como llegan a Loki.

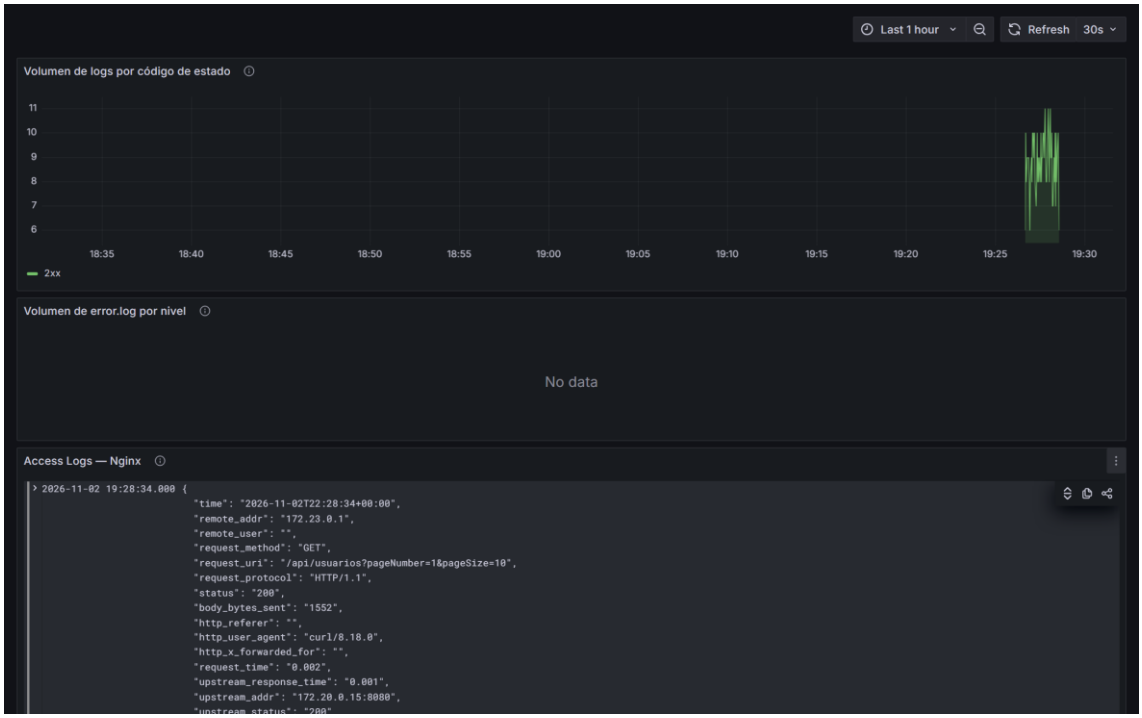


Figura 4: Tablero de logs de Nginx en Grafana con un registro JSON

Fuente: captura del entorno de Argos.

Reglas de detección

La plataforma tiene 18 reglas de alerta (Tabla 11). Las cinco reglas de Loki sobre los logs de Nginx son las que disparan el análisis de Argos; las trece de Prometheus cubren la infraestructura. Cada regla tiene un período `for` que la condición debe sostener antes de pasar a firing, para evitar alertas por picos de pocos segundos.

Tabla 11: Reglas de detección

Origen	Regla	Sev.	Condición
Loki	NginxHighErrorRate	W	5xx sobre el total de requests (ventana 5 min) > 5 % durante 2 min.
Loki	NginxCriticalErrorRate	C	5xx sobre el total > 20 % durante 1 min.
Loki	NginxHighResponseTime	W	p95 de `upstream_response_time` > 2 s durante 3 min.
Loki	NginxSuspiciousIPActivity	W	Una IP con más de 200 requests por minuto durante 2 min.
Loki	NginxErrorLogSpike	W	Más de 1 línea por segundo en el error.log durante 2 min.
Prom.	HostHighCPU / HostCriticalCPU	W / C	CPU del host > 80 % (5 min) / > 95 % (2 min).
Prom.	HostHighMemory	W	Memoria usada > 85 % durante 5 min.
Prom.	HostHighDisk / HostDiskCritical	W / C	Disco usado > 85 % (5 min) / > 95 % (2 min).

Origen	Regla	Sev.	Condición
Prom.	ContainerHighCPU / ContainerHighMemory	W	CPU de un contenedor > 80 % / memoria > 85 % del límite, 5 min.
Prom.	NginxHighActiveConnections	W	Más de 800 conexiones activas durante 5 min.
Prom.	NginxHighRequestRate	W	Más de 500 requests por segundo durante 5 min.
Prom.	NginxWaitingConnections	W	Más de 100 conexiones en espera durante 5 min.
Prom.	NginxDown / NodeExporterDown / CAdvisorDown	C	El exporter no responde (`up == 0`) durante 1 min.

Fuente: elaboración propia.

La Figura 5 muestra las reglas de Prometheus cargadas y evaluándose, agrupadas por archivo y grupo.

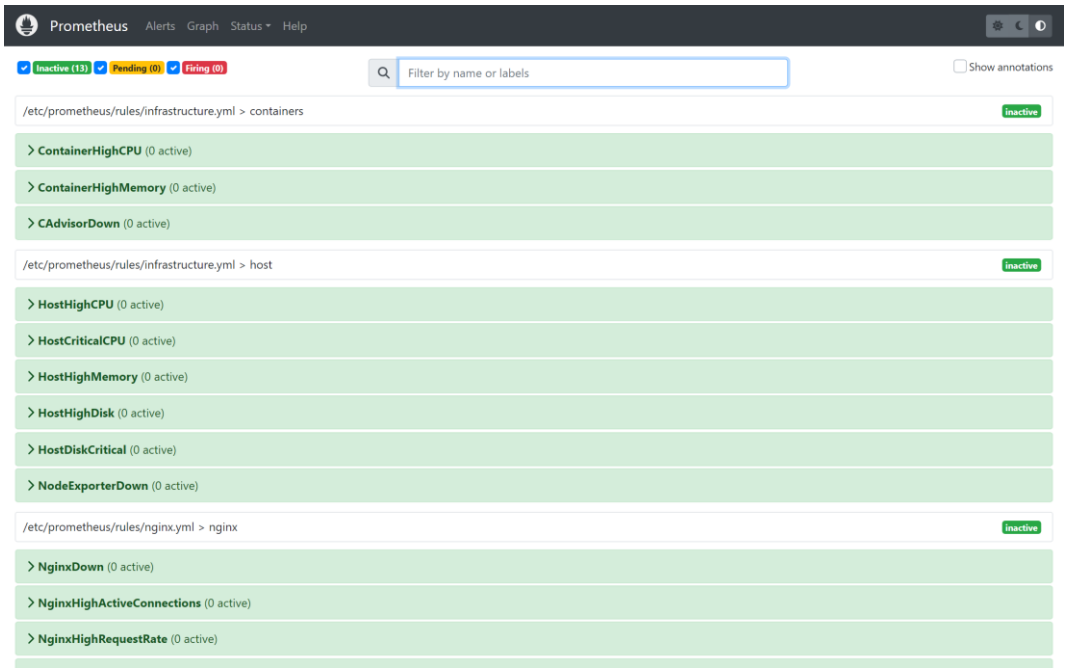


Figura 5: Reglas de alerta cargadas en Prometheus

Fuente: captura del entorno de Argos.

Ruteo de alertas

Alertmanager agrupa las alertas por nombre y severidad, espera 30 segundos antes de enviar el primer grupo para juntar las alertas relacionadas y repite la notificación cada cuatro horas si sigue activa. Las rutas se resumen en la Tabla 12. Las alertas de Loki y todas las críticas van a n8n con repetición cada hora; los warnings de infraestructura van por correo directo, sin análisis. El receptor de n8n también recibe las resoluciones (`send_resolved: true`), que Argos usa para calcular el MTTR. Una regla de inhibición silencia la versión warning de una alerta mientras su versión crítica está activa.

Tabla 12: Rutas de Alertmanager

Coincidencia	Receptor	Repetición
Las cinco alertas de Nginx basadas en logs	webhook-n8n (análisis de Argos)	1 h
Cualquier alerta con `severity=critical`	webhook-n8n	1 h
Resto de alertas `warning`	email-default	4 h
Watchdog	null-receiver (descartada)	-

Fuente: elaboración propia.

Base de conocimiento y búsqueda semántica

Al arrancar, la API crea las colecciones si no existen, crea el usuario administrador inicial y siembra los cinco runbooks manuales. El proceso es idempotente: solo inserta lo que falta. Cada texto pasa por el tokenizer, se trunca a 128 tokens, atraviesa el modelo ONNX y se promedian los vectores de salida para obtener un único embedding de 384 dimensiones.

La consulta (`POST /query`) recibe el texto de la alerta, calcula su embedding y pide a Qdrant los tres incidentes más cercanos por similitud del coseno, considerando solo los que tienen documentación o pasos. Devuelve cada resultado con su puntaje y su clasificación, y la bandera `is_documented` que indica si el mejor supera el umbral vigente. El umbral se lee de la configuración en cada consulta, de modo que un cambio desde el panel tiene efecto en la siguiente alerta.

Flujo inteligente

La Figura 6 muestra el workflow de n8n con sus nodos reales, y la Tabla 13 describe la función de cada uno.

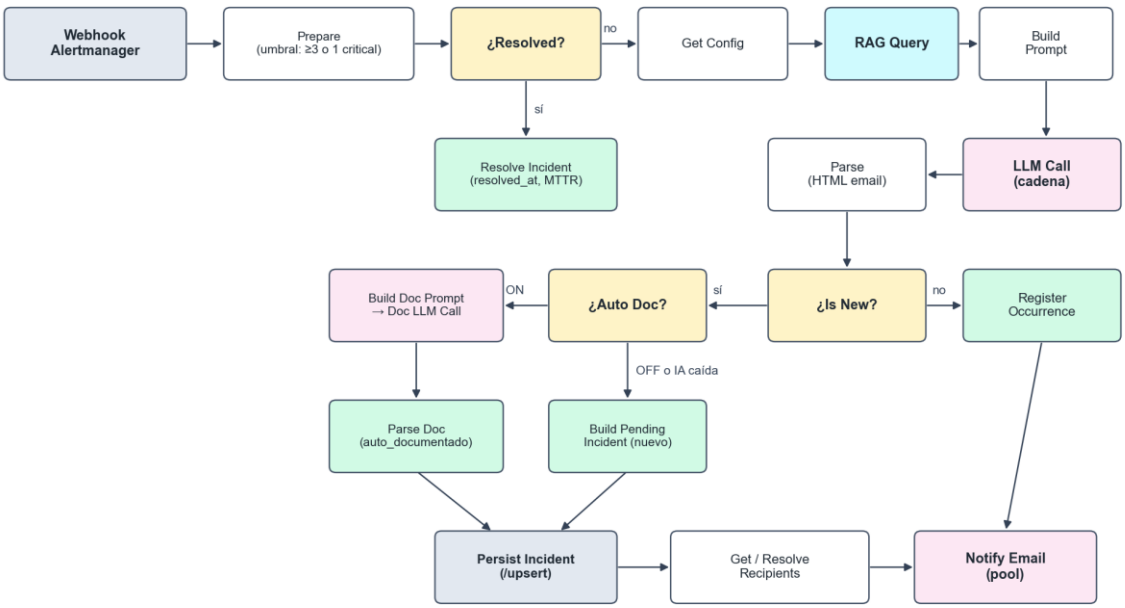


Figura 6: Workflow Alert Analysis en n8n

Fuente: elaboración propia.

Tabla 13: Nodos del workflow Alert Analysis

Nodo	Función
Webhook Alertmanager	Recibe el payload de Alertmanager en `/webhook/argos-alerts`.
Prepare	Extrae alerta, severidad, resumen y descripción; arma el `query_text`. Si la alerta está activa y no hay 3 o más alertas ni una crítica, corta el flujo.
Resolved? / Resolve Incident	Si la alerta terminó, registra `resolved_at` y el MTTR de esa ocurrencia; no analiza ni envía correo.
Get Config	Lee de la API el umbral, la autodocumentación y el modelo vigentes.
RAG Query	Busca incidentes parecidos con `POST /query`.
Build Prompt	Arma el prompt con la alerta, hasta tres incidentes parecidos con su solución y la descripción del stack.
LLM Call / Parse	Llama a `POST /internal/llm/generate` y arma el HTML del correo; si falló toda la cadena, agrega un aviso.
Is New? / Auto Doc?	Decide entre registrar una ocurrencia conocida, autodocumentar o dejar el incidente pendiente de documentar.
Build Doc Prompt / Doc LLM Call / Parse Doc	Genera la documentación en Markdown del incidente nuevo.
Build Pending Incident	Registra el incidente como `nuevo` cuando la autodocumentación está apagada o la IA no respondió.
Register Occurrence	Crea el incidente `pendiente` con la solución del caso conocido.
Persist Incident	Guarda el incidente con `POST /upsert`.

Nodo	Función
Get / Resolve Recipients	Obtiene el pool de destinatarios; si está vacío o falla, usa la variable `ALERT_RECIPIENT_EMAIL`.
Notify Email	Envía el correo con el análisis por SMTP.

Fuente: elaboración propia.

El prompt de análisis pide al modelo que actúe como analista de AIOps, que sintetice el problema y proponga pasos numerados y accionables en español, y que referencie el contexto histórico si aplica. Desde la evaluación del OE3 incluye además una descripción del entorno: un único host con Docker Compose, sin Kubernetes ni systemd, con los nombres reales de los contenedores, para que los comandos propuestos se puedan copiar y ejecutar. Para cada incidente parecido se incluyen sus pasos o, si es un documento aprendido sin pasos, la sección de pasos de resolución del documento (hasta 900 caracteres). El Anexo 4 reproduce un análisis real generado por la plataforma.

Cadena de modelos de lenguaje

La llamada al modelo pasa por la Argos API (`POST /internal/llm/generate`). La API recorre una cadena de proveedores: primero el principal configurado en el panel (`gemini-flash-lite-latest`), luego los eslabones de `LLM_FALLBACK_CHAIN` (Groq con Llama 3.3 70B y OpenRouter con `nvidia/nemotron-3-ultra-550b-a55b:free`). Los eslabones sin clave se saltean. Groq y OpenRouter comparten una implementación común porque ofrecen una API compatible con la de OpenAI.

La salud de cada proveedor se sigue de forma pasiva: cada llamada real registra si respondió o no. Un proveedor que falla con un error 429, un 5xx o un timeout queda 300 segundos en espera al final de la cadena. No hay pings automáticos para no consumir cuota; el administrador puede forzar una prueba con el botón "Probar ahora". Los cambios de estado generan una notificación en el panel, y la respuesta indica qué proveedor respondió y si se usó un respaldo, dato que aparece al pie del correo.

Ciclo de vida del incidente

La Figura 7 muestra los estados de un incidente. Cada ocurrencia se identifica por el tipo de alerta y su hora de inicio, de modo que las repeticiones que envía

Alertmanager no duplican incidentes y la resolución encuentra la ocurrencia exacta.

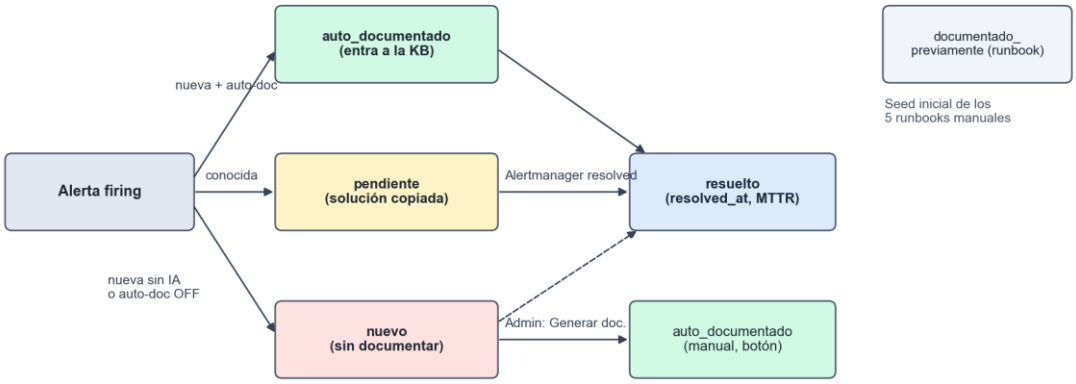


Figura 7: Ciclo de vida de un incidente en Argos

Fuente: elaboración propia.

Según el resultado de la búsqueda, el incidente sigue uno de tres caminos:

- Nuevo: el LLM redacta la documentación técnica (síntoma, causa probable, pasos de resolución y prevención) y el incidente queda como `auto_documentado`, es decir, pasa a formar parte de la base de conocimiento.
- Conocido: se crea un incidente `pendiente` con la solución copiada del documento que coincidió, sin pedirle al modelo que documente de nuevo.
- Falla de toda la cadena de modelos o autodocumentación desactivada: el correo sale igual con un aviso, el incidente queda registrado como "sin documentar" y se genera una notificación en el panel. Un administrador puede documentarlo después con el botón "Generar documentación".

Cuando Alertmanager informa que la alerta terminó, Argos registra la hora de fin y calcula el MTTR real como diferencia entre el fin y el inicio. Un operador también puede marcar un incidente como resuelto desde el panel; en ese caso se usa la hora del clic.

Argos API

La API está escrita en C# sobre .NET 10 y organizada en cinco proyectos (Figura 8). Cada capa depende solo de las inferiores, todo se registra por interfaz con módulos de Autofac y cada tipo vive en su propio archivo.

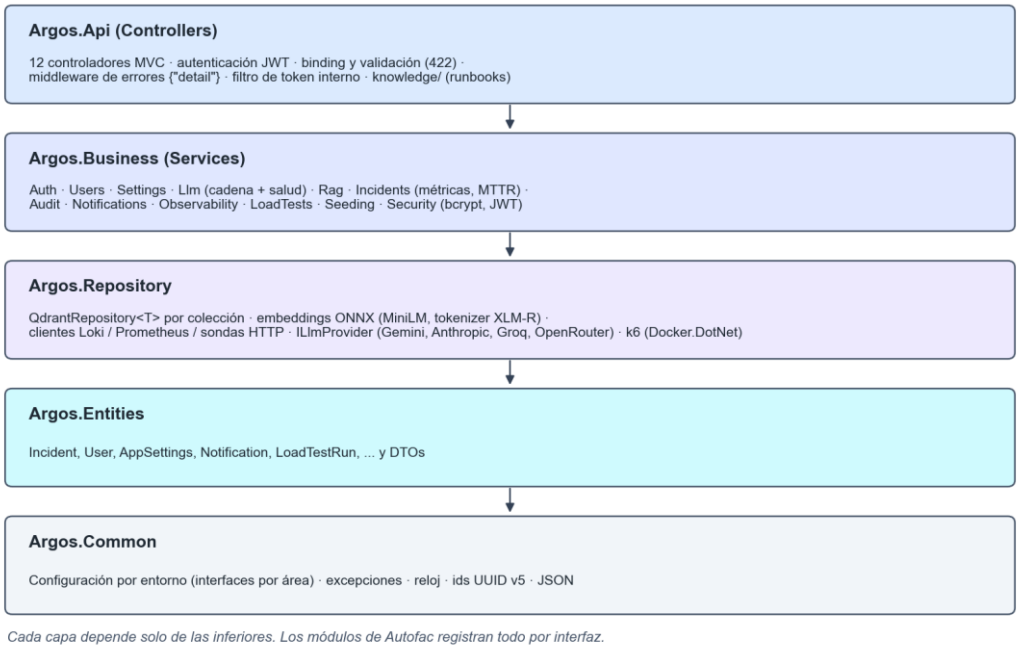


Figura 8: Capas de la Argos API

Fuente: elaboración propia.

La Tabla 14 agrupa los endpoints por área. Los endpoints de usuarios, incidentes, observabilidad, notificaciones y configuración exigen un token JWT; los que usa n8n exigen un token interno en el encabezado `X-Argos-Token` y no son accesibles a través del proxy del panel.

Tabla 14: Endpoints de la Argos API por área

Área	Endpoints	Acceso
Autenticación	`POST /auth/login`, `GET /auth/me`	Público / JWT
Usuarios	`GET`, `POST /users`; `PATCH`, `DELETE /users/{id}`	Administrador
Incidentes	`GET /incidents`, `/incidents/metrics`, `/incidents/knowledge-base`, `/incidents/{id}`, `/{id}/observability`, `/{id}/audit`; `PATCH /incidents/{id}`; `POST /incidents/{id}/document`	JWT (documentar: Administrador)
Observabilidad	`GET /observability/services`, `/observability/nginx-metrics`	JWT
Notificaciones	`GET /notifications`, `POST /notifications/read-all`, `POST /notifications/{id}/read`, `GET /activity`	JWT
Configuración	`GET`, `PATCH /settings`; `GET /integrations/status`	JWT (editar: Administrador)

Área	Endpoints	Acceso
Salud de la IA	`GET /llm/health`, `POST /llm/health/check`	JWT (probar: Administrador)
Pruebas de carga	`GET /loadtests/scenarios`, `/loadtests/runs`, `/runs/{id}`; `POST /loadtests/runs`, `/runs/{id}/stop`	JWT (ejecutar: Administrador)
Uso interno (n8n)	`POST /query`, `/upsert`, `/internal/llm/generate`, `/internal/incidents/occurrence`, `/internal/incidents/resolve`; `GET /internal/config`, `/notifications/recipients`	Token interno
Salud	`GET /health`	Público

Fuente: elaboración propia.

Los errores siguen un contrato uniforme: el código HTTP correspondiente con un cuerpo `{"detail": "..."}`. Un cuerpo o parámetro inválido devuelve 422; una falla del proveedor de LLM al documentar devuelve 502 sin exponer la URL del proveedor (Gemini lleva la clave en la URL); Qdrant caído devuelve 503; y un error no controlado devuelve 500 sin detalles internos. Las contraseñas se guardan con bcrypt de costo 12 y las sesiones usan JWT HS256 con vencimiento de 12 horas.

Perfiles de usuario

Se definieron dos perfiles cuyas funciones se reflejan en la Tabla 15.

Tabla 15: Perfiles de usuario

Perfil	Funciones
Administrador	Iniciar sesión. Consultar todas las vistas. Marcar incidentes como resueltos. Generar la documentación de un incidente con el LLM. Ejecutar y detener pruebas de carga. Crear, editar y eliminar usuarios. Modificar la configuración (proveedor y modelo de IA, umbral de similitud, autodocumentación, destinatarios y frecuencia de actualización). Probar la salud de los modelos.
Operador	Iniciar sesión. Consultar el panel, los incidentes, la base de conocimiento, las métricas y las pruebas de carga. Ver el detalle de un incidente con sus logs y métricas. Marcar incidentes como resueltos. Ver la configuración sin poder modificarla.

Fuente: elaboración propia.

Panel web

El panel es una aplicación React servida por Nginx, que redirige las llamadas `/api` a la Argos API. Tiene siete vistas (Tabla 16) y se actualiza solo con la frecuencia configurada, de 20 segundos por defecto. El encabezado muestra el estado de los modelos de IA y una campana con notificaciones de incidentes, cambios de salud de los proveedores y errores del pipeline. Las capturas de esta sección se tomaron sobre el entorno local con los incidentes acumulados durante la evaluación.

Tabla 16: Vistas del panel de Argos

Vista	Contenido
Dashboard	Indicadores de MTTR (con tendencia de 7 días contra los 7 anteriores), incidentes del día, autodocumentados y pendientes de documentar; incidentes por día, actividad reciente y estado real de la IA.
Incidentes	Tabla paginada con búsqueda y filtros. El detalle muestra los logs de Loki y las métricas de Prometheus de la ventana del incidente, el análisis de la IA, el documento y la auditoría.
Base de conocimiento	Runbooks manuales y documentos autogenerados por el LLM, con búsqueda y filtro por componente.
Métricas	Salud de los servicios mediante sondas HTTP y señales doradas de Nginx desde Loki: solicitudes por código (con 429 como serie propia), latencia p50/p95/p99 y tasa de errores 5xx.
Pruebas de carga	Ejecución bajo demanda de los escenarios k6, visor del código y resultados (p95, tasa de error, checks).
Usuarios	Alta, edición y baja de usuarios con su rol (solo Administrador).
Configuración	Proveedor y modelo de IA (validados contra el proveedor al guardar), umbral, autodocumentación, pool de destinatarios, estado en vivo de las integraciones y salud de los modelos.

Fuente: elaboración propia.

La vista principal (Figura 9) resume el estado operativo. Los indicadores se calculan sobre los incidentes guardados: el MTTR promedio usa solo incidentes con inicio y fin reales, y la tendencia compara los últimos siete días con los siete anteriores. La tarjeta de pendientes de documentar lleva a la lista filtrada.

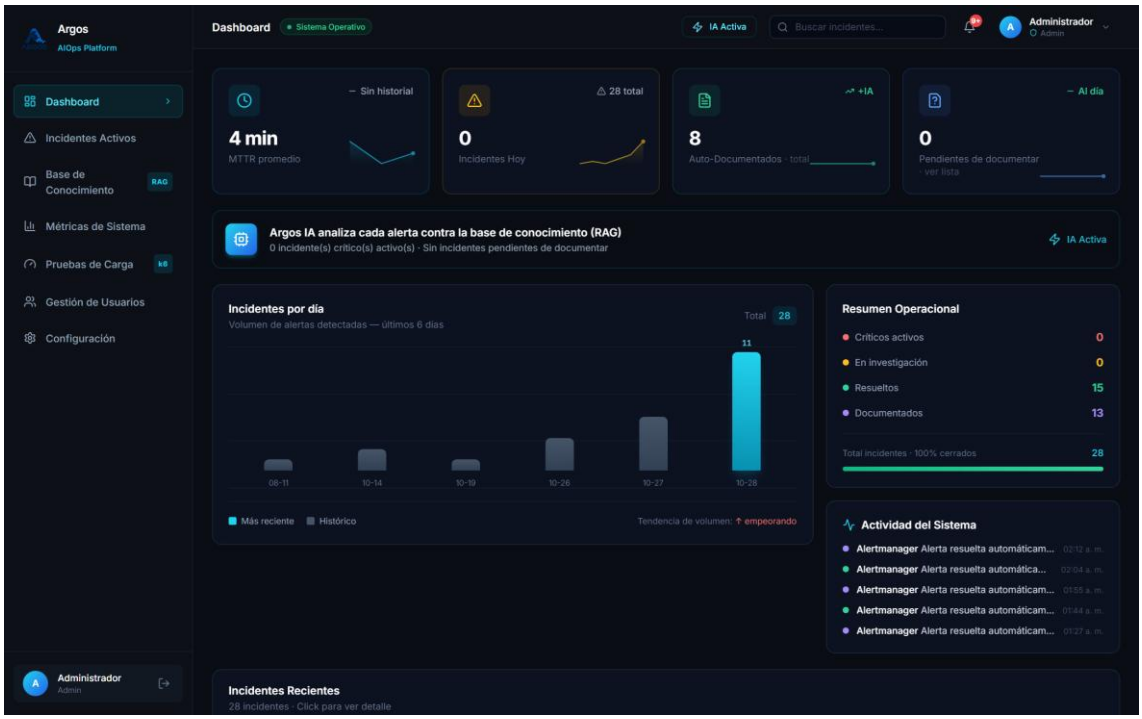


Figura 9: Vista principal del panel de Argos

Fuente: captura del entorno de Argos.

La vista de incidentes (Figura 10) lista las ocurrencias con su severidad, componente, descripción, estado y antigüedad. Al seleccionar una fila se abre el detalle (Figura 11), que muestra el MTTR, las líneas del error log de Nginx que Loki registró en la ventana del incidente, el análisis de Argos con los pasos de resolución y, al pie, la acción disponible según el estado.

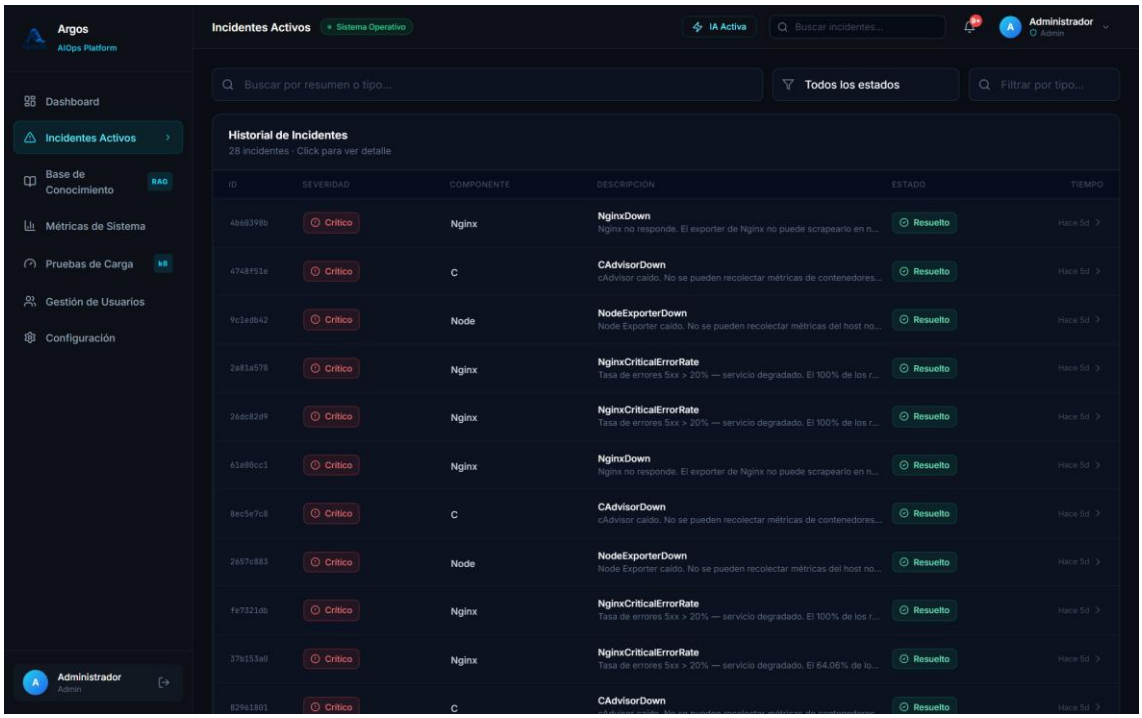


Figura 10: Vista de incidentes

Fuente: captura del entorno de Argos.

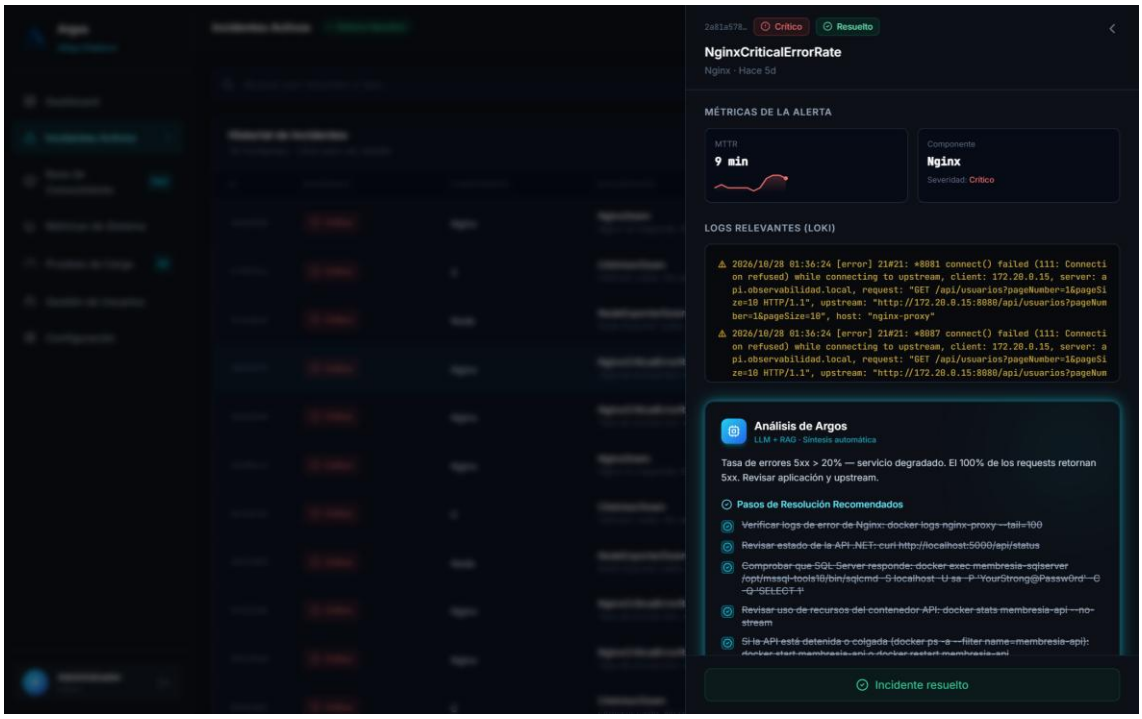


Figura 11: Detalle de un incidente NginxCriticalErrorRate con logs de Loki y análisis

Fuente: captura del entorno de Argos.

La base de conocimiento (Figura 12) reúne los runbooks escritos a mano y los documentos que generó la plataforma. El ejemplo corresponde a un incidente `CAdvisorDown` autodocumentado durante la evaluación: el documento tiene las

secciones pedidas en el prompt y queda disponible para las siguientes ocurrencias.

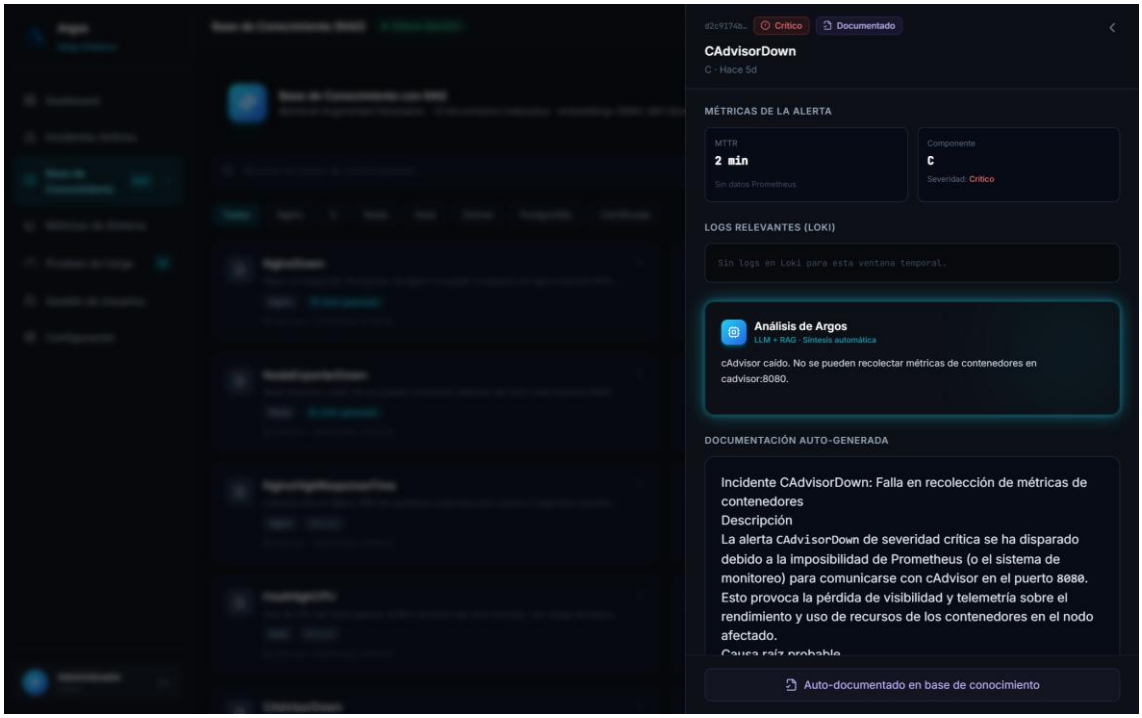


Figura 12: Documento autogenerado en la base de conocimiento

Fuente: captura del entorno de Argos.

La vista de métricas incluye las señales doradas de Nginx calculadas desde Loki (Figura 13): solicitudes por segundo por familia de código de estado, con el 429

del rate limit como serie separada, y latencia del upstream en percentiles 50, 95 y 99. El selector permite ver los últimos 15 minutos, 1, 3 o 6 horas.

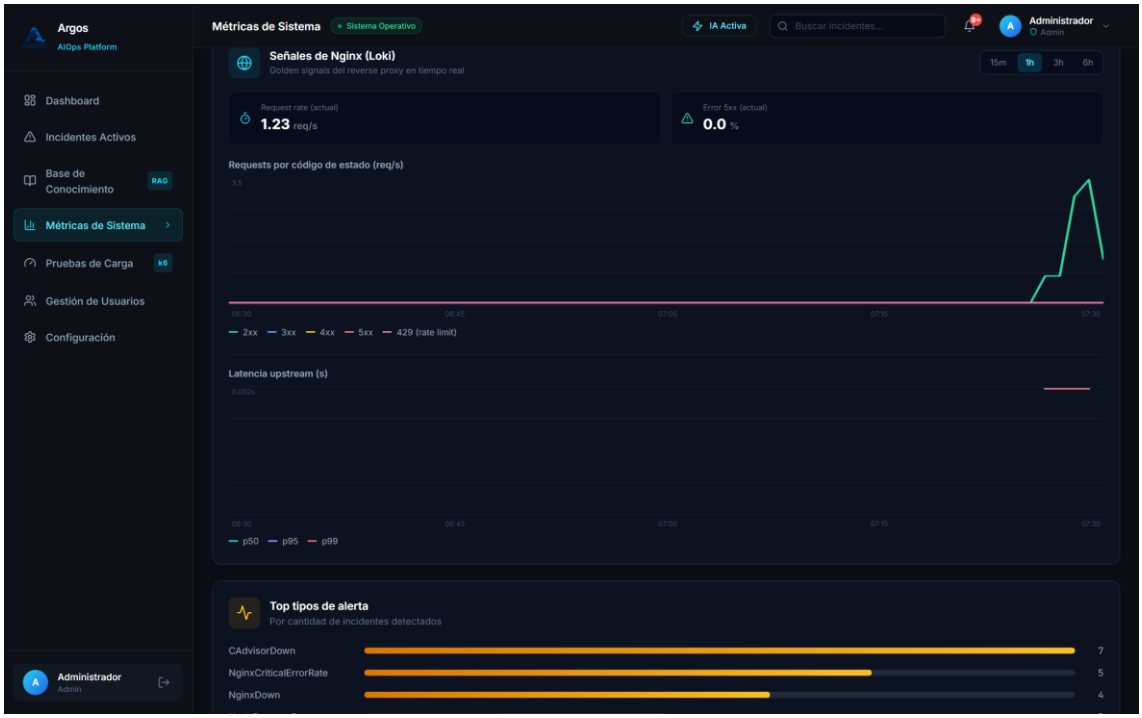


Figura 13: Señales doradas de Nginx en la vista de métricas

Fuente: captura del entorno de Argos.

La vista de pruebas de carga (Figura 14) lista los cuatro escenarios con su perfil y duración, muestra el código del script seleccionado y conserva el historial de corridas con sus métricas. La API lanza un contenedor `grafana/k6` hermano a

través del socket de Docker, permite una sola corrida a la vez (una segunda devuelve 409) y guarda los resultados en la colección `argos_loadtests`.

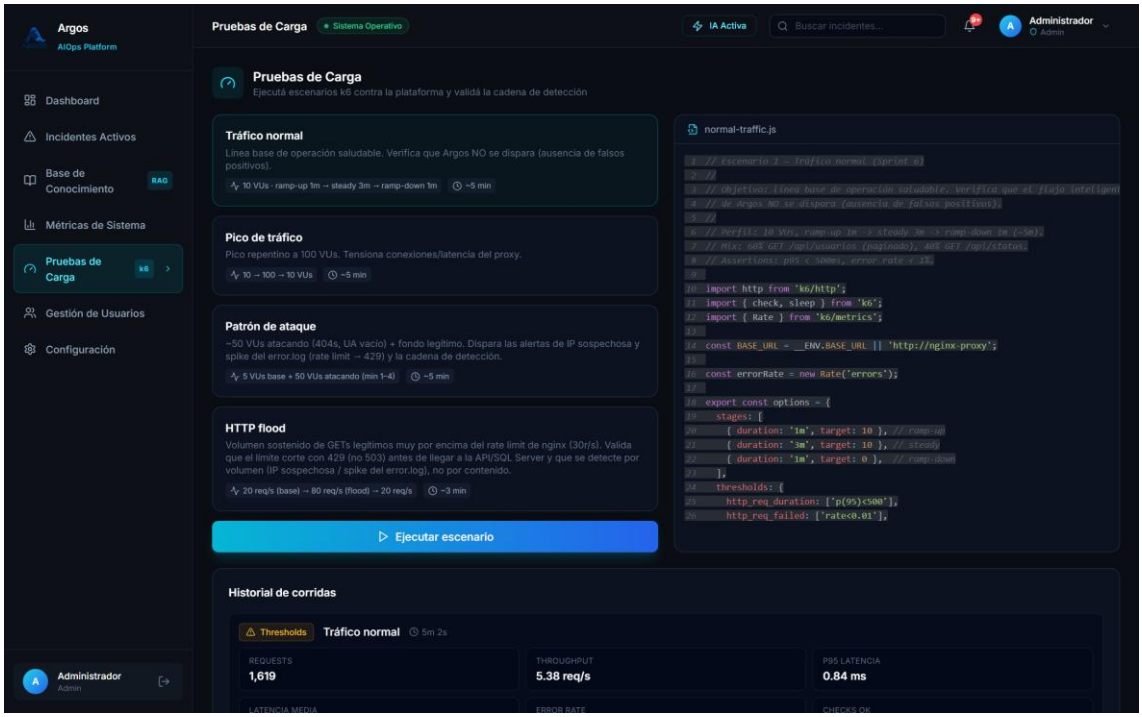


Figura 14: Vista de pruebas de carga con el código del escenario normal

Fuente: captura del entorno de Argos.

La configuración (Figura 15) muestra la cadena de modelos con el estado de cada proveedor, el umbral de similitud, el interruptor de autodocumentación y el pool de destinatarios. Al guardar, la API valida el proveedor y el modelo contra

el servicio real: una clave inválida o un modelo inexistente devuelven 422 y no se aplican.

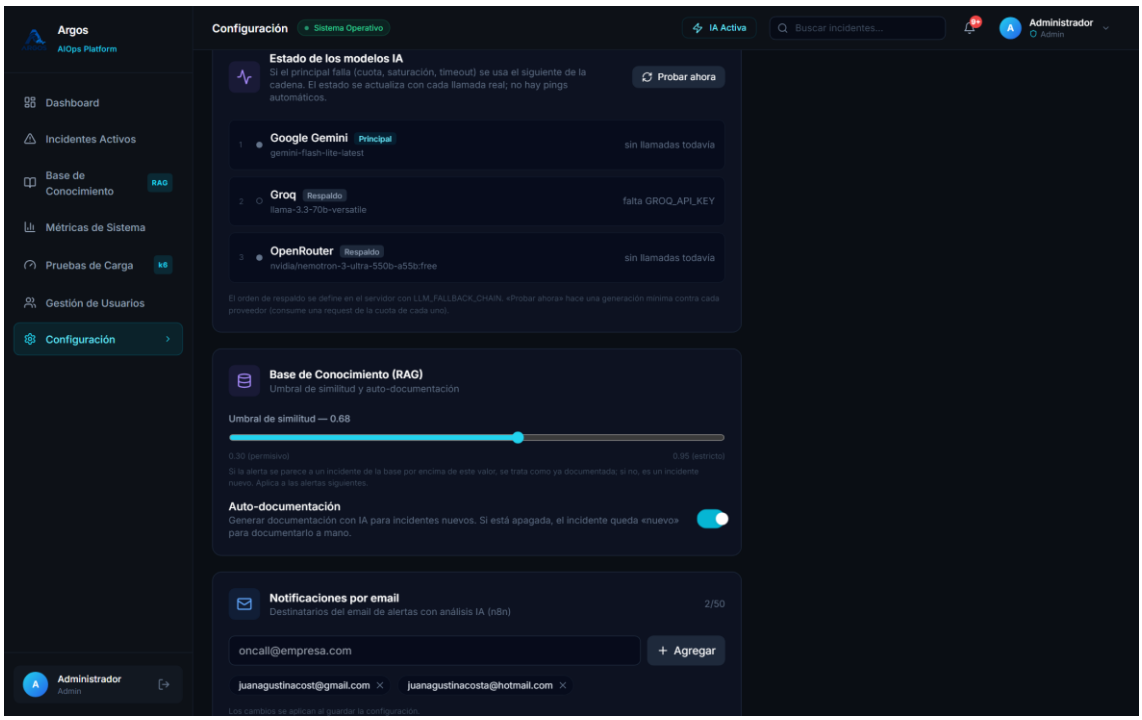


Figura 15: Configuración: modelos de IA, umbral y destinatarios

Fuente: captura del entorno de Argos.

4.5 Validación de la solución tecnológica

La validación cubrió los ejes del tercer objetivo específico: detección, clasificación del RAG, calidad de la propuesta del LLM, persistencia y aprendizaje, y tiempos de resolución. Todas las pruebas se hicieron en el entorno local (Docker Desktop) con el stack completo. Los scripts y los resultados crudos están en las carpetas `k6/` y `eval/` del repositorio.

Detección con escenarios k6

Se ejecutaron cuatro escenarios contra Nginx desde un contenedor k6 dentro de la red de la plataforma (Tabla 17). El escenario normal reparte el tráfico entre el listado paginado de usuarios (60 %) y el estado de la API (40 %). El de ataque combina un fondo de 5 usuarios legítimos durante cinco minutos con una oleada de hasta 50 usuarios entre el minuto 1 y el 4, que piden rutas inexistentes, envían User-Agent vacíos o rotativos y rotan el encabezado X-Forwarded-For.

Tabla 17: Resultados de los escenarios k6

Escenario	Perfil	Resultado
Normal	10 usuarios virtuales, 5 min; 1 625 solicitudes	p95 de 3,23 ms y 0 % de errores. No se disparó ninguna alerta: no hubo falsos positivos.
Pico	Hasta 100 usuarios virtuales; 17 998 solicitudes	La latencia se mantuvo plana (p95 de 2,51 ms). El 61 % de las solicitudes fue rechazado por la limitación de tasa de Nginx. La fase de error duró menos que el `for: 2m` de la regla y no llegó a firing.
Ataque	5 usuarios de base + 50 atacando rutas inexistentes; 28 454 solicitudes	Tasa de 5xx cercana al 74 %. Dispararon NginxCriticalErrorRate, NginxHighErrorRate, NginxErrorLogSpike y NginxSuspiciousIPActivity, que llegaron a n8n. No se autodocumentaron porque ya tenían runbook.
Inundación (flood)	GET legítimos de 20 a 80 solicitudes por segundo desde una IP; 8 099 solicitudes	Nginx cortó 3 027 solicitudes (37 %) con 429, sin errores 5xx y con p95 de 1,86 ms sobre lo que pasó: el backend no se degradó.

Fuente: elaboración propia.

En el escenario de ataque, la distribución de códigos en Loki durante la ventana de cinco minutos fue de 622 respuestas 200, 344 respuestas 400, 4 130 respuestas 404 y 14 822 respuestas 503. Esa prueba se ejecutó cuando Nginx todavía respondía 503 al aplicar el rate limit, y fue la que motivó el cambio a 429: un request frenado por el límite no debe contarse como error del backend. La ausencia de autodocumentación en ese escenario es el comportamiento esperado, porque las alertas disparadas ya tenían runbook y el RAG las clasificó como conocidas.

Catálogo de fallas

Para evaluar el resto de los ejes se armó un catálogo de siete fallas reproducibles, que un script inyecta y restaura (Tabla 18). Cada falla tiene una causa de referencia, el arreglo esperado y la clasificación que debería dar el RAG.

Tabla 18: Catálogo de fallas de la evaluación

Falla	Alerta	Causa inyectada	Arreglo esperado
F1	NginxCriticalErrorRate	API de membresía detenida	`docker start membresia-api`
F2	NginxCriticalErrorRate	SQL Server detenido	`docker start membresia-sqlserver`
F3	NodeExporterDown	node-exporter detenido	`docker start node-exporter`
F4	CAdvisorDown	cAdvisor detenido	`docker start cadvisor`
F5	HostCriticalCPU	Contenedor de estrés de CPU	Identificarlo con `docker stats` y eliminarlo
F6	NginxSuspiciousIPActivity	Una IP con ~1 800 solicitudes por minuto	Limitar o bloquear la IP en Nginx
F7	NginxDown	Exporter de Nginx detenido	`docker start nginx-prometheus-exporter`

Fuente: elaboración propia.

Clasificación del RAG y ajuste del umbral

Con el umbral original de 0,63 el RAG clasificó bien 5 de las 7 fallas (Tabla 19).

Tabla 19: Clasificación del RAG con el umbral original (0,63)

Falla	Mejor puntaje	Mejor coincidencia	Clasificó	Esperado
F1	0,982	NginxCriticalErrorRate	conocida	conocida
F2	0,979	NginxCriticalErrorRate	conocida	conocida
F3	1,000	NodeExporterDown	conocida	conocida
F4	0,460	NodeExporterDown	nueva	nueva
F5	0,789	HostHighCPU	conocida	conocida
F6	0,607	NginxSuspiciousIPActivity	nueva	conocida (error)
F7	0,669	NginxHighResponseTime	conocida	nueva (error)

Fuente: elaboración propia.

Se equivocó en F6, una alerta conocida cuyo texto cambia con la IP y el valor medido, y en F7, una alerta nueva parecida a otra de Nginx. Un barrido entre 0,55 y 0,75 mostró que ningún umbral fijo acierta las siete, porque esos dos casos se cruzan. Se priorizó evitar el falso "conocido", que copia una solución equivocada y no documenta, sobre el falso "nuevo", que solo genera un documento de más. Con 0,68 se aciertan 6 de 7 y desaparecen los falsos "conocido" (Tabla 20). Con siete casos el barrido orienta la decisión pero no es concluyente.

Tabla 20: Barrido del umbral de similitud

Umbral	Aciertos	Falsos "conocido"	Falsos "nuevo"
0,55 a 0,60	6 / 7	1 (F7)	0
0,63 (original)	5 / 7	1 (F7)	1 (F6)
0,68 a 0,75	6 / 7	0	1 (F6)

Fuente: elaboración propia.

Disponibilidad de los modelos

En la primera corrida de evaluación, el modelo principal original (`gemini-flash-latest`) no respondió ninguna de 9 solicitudes: devolvió errores 503 por saturación o agotó el tiempo de 60 segundos; en la segunda corrida tampoco respondió ninguna de 8. Sin la cadena de respaldo, el pipeline no habría producido propuestas (Tabla 21).

Tabla 21: Disponibilidad y latencia de los modelos (corridas 1 y 2)

Modelo	Respuestas válidas	Latencia mediana	Fallas
gemini-flash-latest	0 / 17	-	8 × HTTP 503, 9 × timeout de 60 s
gemini-flash-lite-latest	17 / 17	~1,4 s	-
openrouter/free (enrutador)	15 / 17	~16 a 20 s	2 respuestas "User Safety: safe"
nemotron-3-ultra-550b (fijado)	8 / 8	~13 s	-

Fuente: elaboración propia.

`gemini-flash-lite-latest` respondió todas las solicitudes con una mediana de 1,4 segundos y pasó a ser el principal. Con ese cambio la ejecución del workflow bajó de entre 23 y 68 segundos a entre 5 y 7 segundos. El enrutador gratuito de OpenRouter a veces elegía un clasificador de seguridad que respondía "User Safety: safe" en lugar de un análisis; se probaron otros modelos gratuitos (dos devolvieron 429 por cuota y uno 403) y se fijó `nvidia/nemotron-3-ultra-550b-a55b:free`, que respondió 8 de 8.

Calidad de la propuesta del LLM

Para cada falla se generó el análisis con los dos modelos de la cadena vigente, usando los prompts reales del workflow. Cada respuesta se puntuó de 1 a 5 con la rúbrica del Anexo 6 en diagnóstico, accionabilidad, completitud y claridad, y se contaron las afirmaciones falsas. Se aplicó un criterio explícito: el modelo solo ve la alerta, no la falla, así que en F2, cuya alerta es idéntica a la de F1, proponer revisar la API o la base es un diagnóstico correcto.

La primera evaluación mostró que la calidad dependía más del contexto recuperado que del modelo: con runbook manual la accionabilidad promedio fue 3,60, y con solo documentos autogenerados fue 2,50. La causa era que el prompt pasaba únicamente los pasos de los incidentes parecidos y los documentos aprendidos guardaban la solución en el texto del documento, con la lista de pasos vacía. Los modelos lo decían textualmente: "los incidentes históricos no tienen pasos documentados". Además, sin descripción del entorno proponían comandos de systemd, SSH o Kubernetes. Se corrigió el prompt para incluir un extracto del documento aprendido y una descripción del stack, se corrigió un

runbook que citaba un proyecto de Docker Compose incorrecto y se repitió la evaluación (Figura 16 y Tabla 22).

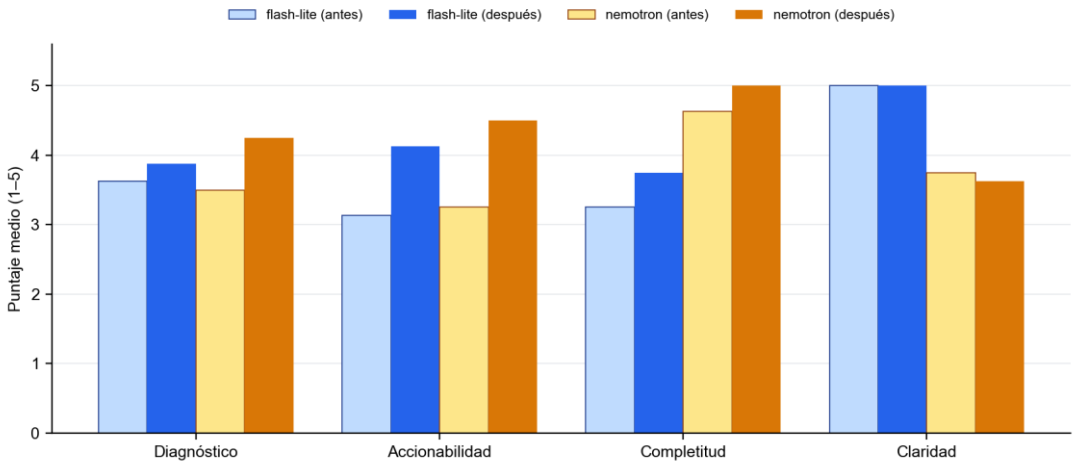


Figura 16: Puntaje medio de la rúbrica antes y después de la corrección del prompt

Fuente: elaboración propia.

Tabla 22: Rúbrica de calidad antes y después de la corrección

Modelo	Diagnóstico	Accionabilidad	Completitud	Claridad	Alucinaciones
gemini-flash-lite (principal)	3,63 → 3,88	3,13 → 4,13	3,25 → 3,75	5,00 → 5,00	4 → 0
nemotron (respaldo)	3,50 → 4,25	3,25 → 4,50	4,63 → 5,00	3,75 → 3,63	7 → 9

Fuente: elaboración propia.

Tabla 23: Efecto de la corrección según el contexto disponible

Contexto del RAG	Diagnóstico	Accionabilidad
Con runbook manual (F1, F2, F5, F6)	3,90 → 4,00	3,60 → 4,20
Solo documentos autogenerados (F3, F4, F7)	3,00 → 4,17	2,50 → 4,50

Fuente: elaboración propia.

La mejora fue mayor donde solo había conocimiento aprendido: los modelos pasaron a citar la solución histórica ("la solución histórica asume systemd; traducida a Docker") y a proponer el arreglo exacto con los nombres reales de los contenedores. El caso extremo fue nemotron en F4, que pasó de 1 a 5 en accionabilidad: antes resolvía todo con Kubernetes. flash-lite quedó sin alucinaciones y con claridad máxima. nemotron es más completo, con verificaciones y ramas de decisión, pero sus respuestas largas todavía inventan detalles (métricas, puertos, nombres de red), por lo que se mantiene como respaldo. F7 siguió siendo el caso más difícil: los dos modelos sospechan primero de Nginx, aunque el procedimiento termina revisando el exporter y llega al arreglo.

Persistencia y aprendizaje

Para comprobar que la plataforma aprende, se disparó dos veces una alerta que no estaba en la base (Tabla 24).

Tabla 24: Prueba de aprendizaje con dos ocurrencias de la misma alerta

Falla y ocurrencia	Puntaje RAG	Rama del workflow	Estado final	Documento
F4, 1.ª (umbral 0,63)	0,398	Nueva → autodocumentación	`auto_documentado`	Generado por el LLM
F4, 2.ª	0,968	Conocida → Register Occurrence	`pendiente`	Copiado (1 958 caracteres), sin LLM
F7, 1.ª (umbral 0,68)	0,669 contra otra alerta	Nueva → autodocumentación (6,7 s)	`auto_documentado`	Generado por el LLM
F7, 2.ª	0,987	Conocida → Register Occurrence (5,1 s)	`pendiente` → `resuelto`	Copiado (1 829 caracteres)

Fuente: elaboración propia.

La misma alerta que la primera vez no tenía coincidencia pasó a reconocerse con 0,968 contra el documento que había generado la propia plataforma, y la segunda vez no se usó el modelo para documentar. Con el umbral original, F7 habría recibido el runbook de latencia de Nginx, que no corresponde; con 0,68 se documentó bien y la siguiente vez se reconoció. Antes de la corrección del texto guardado, la misma alerta obtenía 0,398 contra su propio documento y se volvía a documentar en cada repetición.

Tiempos de resolución

El protocolo original preveía un experimento con un operador humano, alternando corridas con y sin Argos y con las fallas inyectadas en modo ciego para que el operador no supiera cuál era. Se hizo una primera corrida con Argos (F4: propuesta a 1,0 minuto, diagnóstico a 2,4 minutos y MTTR de 4,1 minutos) y el experimento se canceló por decisión del autor. Se reemplazó por diez corridas automáticas (F1, F2, F3, F4 y F7, dos veces cada una). Un script inyecta la falla, espera la alerta, lee el análisis que salió por correo, verifica si contiene el arreglo exacto, lo aplica como lo haría un operador que sigue la propuesta y espera la resolución en Alertmanager (Tabla 25). La Figura 17 ubica estos tiempos en una línea de tiempo.

Tabla 25: Corridas automáticas (minutos, promedio de dos corridas por falla)

Falla	Detección	Propuesta	¿Arreglo exacto?	Hasta el arreglo	MTTR
F1 API caída	2,92	1,09	Sí / Sí	1,22	9,54
F2 SQL caído	2,98	1,09	Parcial / Parcial	1,20	10,03
F3 node-exporter	1,94	1,10	Sí / Sí	1,19	2,03
F4 cAdvisor	2,10	1,08	Sí / Sí	1,17	2,00
F7 exporter Nginx	1,69	1,09	Sí / Sí	1,19	2,03

Fuente: elaboración propia. Detección: de la inyección a la alerta. Propuesta y arreglo: desde el inicio de la alerta (startsAt).

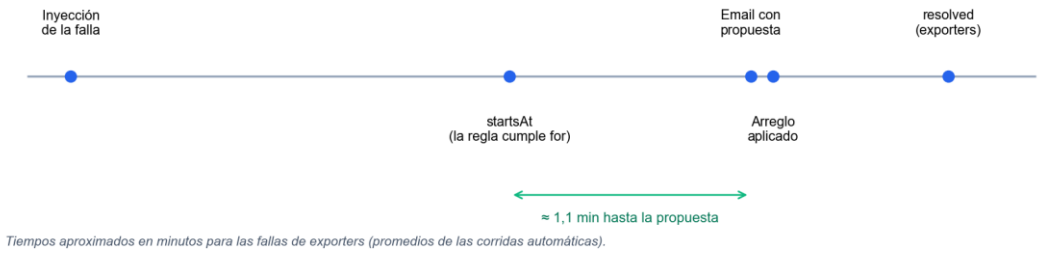


Figura 17: Línea de tiempo de una corrida típica

Fuente: elaboración propia.

Argos entregó la propuesta en 1,1 minutos desde el inicio de la alerta en las diez corridas, siempre con el modelo principal. Casi todo ese tiempo corresponde a la entrega de Alertmanager (30 segundos de agrupamiento más la evaluación de la regla); n8n procesa en unos 6 segundos. En 8 de las 10 corridas la propuesta contenía el arreglo exacto. En F2 la alerta es idéntica a la de F1: la propuesta incluyó el chequeo con `sqlcmd` que revela la caída de la base, pero solo sugirió reiniciar la API. Es un diagnóstico correcto y un arreglo incompleto.

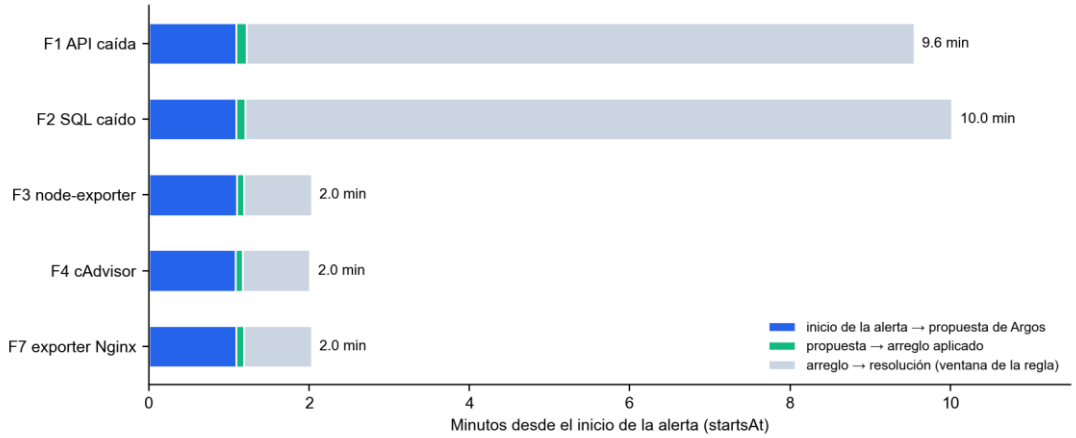


Figura 18: Composición del MTTR siguiendo la propuesta de Argos

Fuente: elaboración propia.

Como muestra la Figura 18, el MTTR siguiendo la propuesta fue de unos 2 minutos en las fallas de exporters y de unos 10 en las de errores 5xx. En estas últimas, cerca de 8,8 minutos corresponden a que la regla deje de cumplirse después del arreglo, porque calcula la tasa sobre una ventana de 5 minutos que tiene que limpiarse. Ese piso lo impone la regla y es igual con o sin Argos. El indicador comparable con un proceso manual es el tiempo hasta la acción correcta, cerca de 1,2 minutos: un operador sin Argos tendría que llegar al mismo arreglo partiendo solo de la alerta.

Pruebas del panel

Se escribió una suite E2E con Playwright que recorre las siete vistas del panel y la API contra el stack real, con un administrador y un operador creados para la prueba. La primera corrida completa ejecutó 69 pruebas: 55 pasaron, 13 fallaron y 1 se omitió. Las fallas se clasificaron por prioridad (Tabla 26).

Tabla 26: Hallazgos de la suite E2E del panel

ID	Hallazgo	Prioridad	Estado
F-01	Endpoints de incidentes y actividad accesibles sin autenticación; el PATCH anónimo no se auditaba.	Alta	Corregido
F-08	Acción de documentar disponible para cualquier rol.	Alta	Corregido
F-13	Endpoints internos de n8n expuestos sin token; con `/upsert` se podían escribir incidentes falsos en la base.	Alta	Corregido
F-07	La base de conocimiento mostraba el umbral fijo en 0,63.	Media	Corregido
F-09	Título incorrecto en la vista de pruebas de carga.	Baja	Corregido
F-02	Marcar como resuelto no refresca la lista de incidentes.	Media	Abierto
F-03	Un error del LLM al documentar no se muestra en la interfaz.	Media	Abierto
F-04	Los runbooks manuales no muestran el cuerpo del documento.	Media	Abierto
F-05	El filtro por tipo exige el nombre exacto de la alerta.	Media	Abierto
F-06	El botón "Nuevo documento" no tiene funcionalidad.	Media	Abierto

Fuente: elaboración propia.

La falla F-13 era la más grave: permitía envenenar la base de conocimiento con documentos falsos que después el RAG le pasaría al modelo. Se corrigió exigiendo el token interno en esos endpoints, bloqueándolos en el proxy del panel y publicando el puerto de la API solo en la interfaz local. La verificación posterior con una alerta simulada confirmó que el workflow seguía funcionando con el token.

Limitaciones de la validación

- No se midió una línea base manual de MTTR, por decisión del autor. No hay datos propios para calcular un porcentaje de reducción; se informa el tiempo hasta la propuesta correcta (1,1 min) y hasta la acción correcta (1,2 min).
- El operador de las corridas automáticas es simulado: aplica el arreglo en cuanto llega la propuesta. Una persona sumaría el tiempo de leerla y ejecutarla.
- La rúbrica de calidad la puntuó un LLM (Claude) con revisión del autor, no evaluadores humanos independientes.
- Las muestras son chicas (10 corridas, 7 escenarios) y las fallas son simples, de causa única y con arreglo de un comando.
- El entorno es local; los tiempos de detección dependen de los parámetros de las reglas y de Alertmanager. k6 genera el tráfico desde una sola IP, así que simula un ataque de denegación de servicio, no uno distribuido.
- No hay detección por contenido de los requests: un escaneo lento o un intento de inyección solo se ven si producen errores en volumen suficiente para las reglas existentes.

Grado de cumplimiento de los objetivos

Tabla 27: Cumplimiento de los objetivos específicos

Objetivo	Cumplimiento	Pendiente
OE1. Arquitectura de observabilidad como código y reglas de detección	85 %	Despliegue en un proveedor de nube (GCP o AWS) adaptando el provider de Terraform.
OE2. Capa inteligente: RAG, orquestación, LLM y autodocumentación	100 %	Sin pendientes.
OE3. Validación con k6, calidad del LLM y persistencia	95 %	Línea base manual de MTTR (limitación declarada).

Fuente: elaboración propia.

Capítulo 5. Conclusiones y futuros trabajos

El proyecto cumplió el objetivo general: Argos detecta incidentes a partir de umbrales, genera con RAG y un modelo de lenguaje una síntesis con pasos de resolución y documenta en forma automática los incidentes que no estaban registrados. La validación mostró que la plataforma entrega una propuesta en alrededor de un minuto desde el inicio de la alerta y que, en las fallas del catálogo, esa propuesta suele contener el comando exacto que resuelve el problema.

Conclusiones por objetivo

El primer objetivo quedó cumplido en el entorno local. La infraestructura está definida como código, se levanta con un solo comando y tiene 18 reglas que cubren la experiencia del usuario del servicio (errores, latencia, tráfico anómalo) y el estado de la infraestructura. La validación con k6 mostró que el tráfico normal no dispara alertas y que un ataque recorre la cadena completa hasta n8n. Falta la migración a un proveedor de nube, que la definición en Terraform deja preparada.

El segundo objetivo es el centro del trabajo y quedó cumplido. La base de conocimiento vectorial, el flujo de orquestación, la cadena de modelos y la autodocumentación funcionan de punta a punta, también cuando el proveedor principal falla. El ciclo de vida de las alertas permite medir el MTTR real de cada ocurrencia, y el panel hace visible y configurable todo el proceso.

El tercer objetivo se cumplió con una limitación declarada. Se midieron la detección, la clasificación del RAG, la calidad de las propuestas, el aprendizaje y los tiempos hasta la propuesta y la acción correcta. No se midió el tiempo de un operador humano sin la plataforma, por lo que el informe no afirma un porcentaje de reducción del MTTR.

Aportes del PFC a la solución del problema

El aporte central es que Argos aprende. Una alerta nueva genera un documento que la siguiente ocurrencia reconoce con una similitud cercana a 0,97, sin volver a gastar el modelo, y la calidad de las respuestas mejora cuando el prompt aprovecha ese conocimiento. Con eso la plataforma reduce la dependencia del

conocimiento de personas puntuales y conserva lo aprendido en cada incidente, que era la brecha identificada en el estado del arte.

La evaluación dejó además algunas lecciones prácticas. El contexto que recupera el RAG pesó más en la calidad que la elección del modelo. Un umbral de similitud fijo tiene casos que no puede separar, y conviene calibrarlo según el costo de cada tipo de error. Depender de un único proveedor de LLM no es viable: el modelo principal original falló todas las solicitudes durante la evaluación, y la cadena de respaldo fue la que sostuvo el pipeline. Por último, describir el entorno real en el prompt fue la corrección más barata y la de mayor efecto: eliminó las respuestas pensadas para otra infraestructura.

Adquisición o profundización de temas de la disciplina

El trabajo exigió integrar en un mismo sistema temas que la carrera aborda por separado: redes y HTTP, contenedores, infraestructura como código, observabilidad, arquitectura de software en capas, bases de datos vectoriales y modelos de lenguaje. También obligó a diseñar una evaluación con métricas propias (rúbrica, barrido de umbral, corridas automáticas) y a reportar sus límites con precisión. En lo personal, el mayor aprendizaje fue que en un sistema que depende de servicios externos la mayor parte del esfuerzo está en los casos de falla: qué pasa cuando el proveedor no responde, cuando el texto no coincide o cuando una alerta se repite.

Trabajos futuros

- Desplegar la plataforma en GCP o AWS reutilizando el código de Terraform y medir latencia y tasa de error en ese entorno.
- Realizar un estudio con varios operadores humanos, en condiciones con y sin Argos, para estimar la reducción del MTTR. Las herramientas para ese experimento (inyección en modo ciego, cronómetro contra Alertmanager y registro de hitos) ya están en el repositorio.
- Sumar la coincidencia exacta del tipo de alerta como señal adicional a la similitud, para resolver casos como F6.
- Enmascarar direcciones IP antes de enviar texto a proveedores externos y fijar políticas de retención de logs.

- Incorporar detección por contenido de los requests (un WAF delante de Nginx o reglas de Loki por patrón) para escaneos e intentos de inyección.
- Exponer métricas propias de la Argos API y de Qdrant con OpenTelemetry, e incorporar integración continua con GitHub Actions.
- Resolver las observaciones abiertas del panel (F-02 a F-06) y regenerar los documentos autogenerados antes de la corrección del prompt, que todavía mencionan systemd.

Referencias

[1] G. Yu, G. Mai, R. Wang, R. Li, P. Chen, L. Pan y R. Xu, «AlertGuardian: Intelligent Alert Life-Cycle Management for Large-scale Cloud Systems,» *arXiv preprint arXiv:2601.14912,* enero 2026.

[2] C. Albuquerque y F. F. Correia, «Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications,» *arXiv preprint arXiv:2510.02991,* octubre 2025.

[3] A. Hrusto, N. bin Ali, E. Engström y Y. Wang, «Monitoring Data for Anomaly Detection in Cloud-Based Systems: A Systematic Mapping Study,» *ACM Transactions on Software Engineering and Methodology,* vol. 35, nº 4, pp. 1-36, 2026.

[4] T. Cui, S. Ma, Z. Chen, T. Xiao, S. Tao, Y. Liu, S. Zhang, D. Lin, C. Liu, Y. Cai, W. Meng, Y. Sun y D. Pei, «LogEval: A Comprehensive Benchmark Suite for Large Language Models in Log Analysis,» *arXiv preprint arXiv:2407.01896,* julio 2024.

[5] Prometheus Authors, «Prometheus documentation,» [En línea]. Available: https://prometheus.io/docs/. [Último acceso: 3 noviembre 2026].

[6] Grafana Labs, «Grafana Loki documentation,» [En línea]. Available: https://grafana.com/docs/loki/latest/. [Último acceso: 3 noviembre 2026].

[7] Prometheus Authors, «Alertmanager,» [En línea]. Available: https://prometheus.io/docs/alerting/latest/alertmanager/. [Último acceso: 3 noviembre 2026].

[8] B. Beyer, C. Jones, J. Petoff y N. R. Murphy, Edits., Site Reliability Engineering: How Google Runs Production Systems, Sebastopol, CA, USA: O'Reilly Media, 2016.

[9] Y. Remil, A. Bendimerad, R. Mathonat y M. Kaytoue, «AIOps Solutions for Incident Management: Technical Guidelines and A Comprehensive Literature Review,» *arXiv preprint arXiv:2404.01363,* abril 2024.

[10] P. Gupta, K. Bhukar, H. Kumar, S. Nagar, P. Mohapatra y D. Kar, «Scalable and Efficient Large-Scale Log Analysis with LLMs: An IT Software Support Case Study,» *arXiv preprint arXiv:2511.14803,* noviembre 2025.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel y D. Kiela, «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» de *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[12] N. Reimers y I. Gurevych, «Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation,» de *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[13] Qdrant, «Qdrant documentation,» [En línea]. Available: https://qdrant.tech/documentation/. [Último acceso: 3 noviembre 2026].

[14] Y. A. Malkov y D. A. Yashunin, «Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,» *IEEE Transactions on Pattern Analysis and Machine Intelligence,* vol. 42, nº 4, pp. 824-836, abril 2020.

[15] M. Hron y N. Obwegeser, «Why and How is Scrum Being Adapted in Practice: A Systematic Review,» *Journal of Systems and Software,* vol. 183, p. 111110, enero 2022.

[16] HashiCorp, «Terraform documentation,» [En línea]. Available: https://developer.hashicorp.com/terraform/docs. [Último acceso: 3 noviembre 2026].

[17] LocalStack, «LocalStack documentation,» [En línea]. Available: https://docs.localstack.cloud/. [Último acceso: 3 noviembre 2026].

[18] Docker Inc., «Docker Compose documentation,» [En línea]. Available: https://docs.docker.com/compose/. [Último acceso: 3 noviembre 2026].

[19] Microsoft, «.NET documentation,» [En línea]. Available: https://learn.microsoft.com/dotnet/. [Último acceso: 3 noviembre 2026].

[20] Grafana Labs, «Grafana Alloy documentation,» [En línea]. Available: https://grafana.com/docs/alloy/latest/. [Último acceso: 3 noviembre 2026].

[21] Microsoft, «ONNX Runtime documentation,» [En línea]. Available: https://onnxruntime.ai/docs/. [Último acceso: 3 noviembre 2026].

[22] Google, «Gemini API documentation,» [En línea]. Available: https://ai.google.dev/gemini-api/docs. [Último acceso: 3 noviembre 2026].

[23] Groq, «GroqCloud documentation,» [En línea]. Available: https://console.groq.com/docs. [Último acceso: 3 noviembre 2026].

[24] OpenRouter, «OpenRouter documentation,» [En línea]. Available: https://openrouter.ai/docs. [Último acceso: 3 noviembre 2026].

[25] n8n GmbH, «n8n documentation,» [En línea]. Available: https://docs.n8n.io/. [Último acceso: 3 noviembre 2026].

[26] Meta Open Source, «React documentation,» [En línea]. Available: https://react.dev/. [Último acceso: 3 noviembre 2026].

[27] Grafana Labs, «Grafana k6 documentation,» [En línea]. Available: https://grafana.com/docs/k6/latest/. [Último acceso: 3 noviembre 2026].

[28] Microsoft, «Playwright documentation,» [En línea]. Available: https://playwright.dev/docs/intro. [Último acceso: 3 noviembre 2026].

[29] G. Greco, «A Comparative Analysis of Load Testing Tools for Microservice Architectures,» *IEEE Access,* vol. 8, pp. 196354-196375, 2020.

[30] Association for Computing Machinery, «ACM Code of Ethics and Professional Conduct,» 2018. [En línea]. Available: https://www.acm.org/code-of-ethics. [Último acceso: 3 noviembre 2026].

[31] República Argentina, «Ley 25.326 de Protección de los Datos Personales,» 2 noviembre 2000. [En línea]. Available: https://www.argentina.gob.ar/normativa/nacional/ley-25326-64790. [Último acceso: 3 noviembre 2026].

[32] M. Guerriero, M. Garriga, D. A. Tamburri y F. Palomba, «Adoption, Support, and Challenges of Infrastructure-as-Code: Insights from Industry,» de *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Cleveland, OH, USA, 2019.

Anexos

Anexo 1. Puesta en marcha del entorno

La plataforma se levanta en una máquina con Docker con los siguientes pasos, documentados en el `README.md` del repositorio. El equipo anfitrión recomendado tiene al menos 8 GB de RAM, 4 núcleos y 60 GB de almacenamiento.

1. Crear la red compartida una sola vez: `docker network create argos-net`.
2. Copiar `docker/monitoring/.env.example` a `.env` y completar las claves de los proveedores de LLM, la credencial SMTP, la clave de cifrado de n8n y el token interno.
3. Levantar todos los stacks desde la raíz: `docker compose up -d --build`.
4. Importar y activar el workflow de n8n dentro del contenedor `argos-n8n`: `n8n import:workflow --input=/workflows/alert-analysis.json` y `n8n update:workflow --id=argos-alert-analysis --active=true`; luego reiniciar el contenedor y adjuntar la credencial SMTP al nodo de correo.
5. Ingresar al panel en `http://localhost:3001` con el usuario administrador inicial y cambiar su contraseña.

Para verificar el despliegue se pueden consultar Prometheus en `:9090`, Grafana en `:3000`, Alertmanager en `:9093`, Qdrant en `:6333/readyz` y la API de ejemplo a través de Nginx en `http://localhost/api/status`.

Anexo 2. Variables de entorno

Tabla 28: Variables de entorno principales

Variable	Función
`GEMINI_API_KEY`, `GROQ_API_KEY`, `OPENROUTER_API_KEY`, `ANTHROPIC_API_KEY`	Claves de los proveedores de LLM. Un eslabón sin clave se saltea.
`LLM_PROVIDER`, `GEMINI_MODEL`, `GROQ_MODEL`, `OPENROUTER_MODEL`, `CLAUDE_MODEL`	Proveedor y modelos por defecto hasta que se elijan otros en el panel.
`LLM_FALLBACK_CHAIN`	Cadena de respaldo en orden, como `proveedor:modelo` separados por comas.
`LLM_COOLDOWN_SECONDS`	Segundos que un proveedor fallido queda al final de la cadena (300 por defecto).
`SIMILARITY_THRESHOLD`	Umbral de similitud inicial (el vigente se configura en el panel).
`ALERT_THRESHOLD`	Cantidad mínima de alertas para analizar un grupo sin alertas críticas (3).
`ALERT_RECIPIENT_EMAIL`	Destinatario de respaldo si el pool está vacío.
`ARGOS_INTERNAL_TOKEN`	Token compartido entre la API y n8n para los endpoints internos.

Variable	Función
N8N_ENCRYPTION_KEY	Clave con la que n8n cifra sus credenciales.
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM	Servidor de correo para las notificaciones.
ARGOS_AUTO_SEED, ARGOS_SEED_DEMO	Carga de runbooks al arrancar e incidentes de demostración (apagada).
RAG_API_URL, CORS_ORIGINS, DASHBOARD_PORT	URL interna de la API, orígenes permitidos y puerto del panel.

Fuente: elaboración propia.

Anexo 3. Ejemplo de runbook manual

Los runbooks se escriben en Markdown con un encabezado YAML. El siguiente es el runbook de errores 5xx que se carga al arrancar (la contraseña de desarrollo se reemplazó por asteriscos):

```
---
tipo_alerta: NginxHighErrorRate
resumen: "Alta tasa de errores HTTP 5xx en Nginx (>5% de las requests durante 5 minutos)"
pasos:
  - "Verificar logs de error de Nginx: docker logs nginx-proxy --tail=100"
  - "Revisar estado de la API .NET: curl http://localhost:5000/api/status"
  - "Comprobar que SQL Server responde: docker exec membresia-sqlserver /opt/mssql-
tools18/bin/sqlcmd -S localhost -U sa -P '********' -C -Q 'SELECT 1'"
  - "Revisar uso de recursos del contenedor API: docker stats membresia-api --no-stream"
  - "Si la API está detenida o colgada (docker ps -a --filter name=membresia-api): docker
start membresia-api o docker restart membresia-api"
  - "Si el problema persiste, revisar los logs de la aplicación en Grafana (dashboard
nginx-logs)"
estado: documentado_previamente
fuente: manual
---

## Contexto

La alerta `NginxHighErrorRate` se dispara cuando la tasa de errores 5xx supera el 5% en
una
ventana de 5 minutos. Los errores 5xx en Nginx indican que las requests llegan al proxy
pero
fallan en el upstream (API o base de datos).

## Causas comunes

- La API .NET no responde o está caída (connection refused / timeout)
- SQL Server no está disponible o la BD no está inicializada
- Falta de recursos del host (CPU/RAM)
- Errores no manejados en la aplicación

## Criterio de resolución

La alerta se resuelve sola cuando el error rate vuelve a < 5% durante 5 minutos.
```

Anexo 4. Ejemplo de análisis generado

Análisis generado por `gemini-flash-lite-latest` para la falla F1 durante la re-evaluación del 27 de octubre de 2026 (2,1 segundos). Es el texto que recibe el operador en el correo:

```
**Síntesis del Problema:**
```

````
Falla crítica en el entorno Argos: el reverse proxy (`nginx-proxy`) reporta que el 87.5% de las solicitudes HTTP retornan errores 5xx (tasa > 20%), indicando una interrupción del servicio o un fallo en el upstream (`membresia-api`).

**Pasos de Resolución:**

1. Verificar logs de error de Nginx:
   ```bash
   docker logs nginx-proxy --tail=100
   ```
2. Revisar el estado de la API .NET:
   ```bash
   curl http://localhost:5000/api/status
   ```
3. Comprobar que SQL Server responde:
   ```bash
   docker exec membresia-sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P '********' -C -Q 'SELECT 1'
   ```
4. Revisar uso de recursos del contenedor de la API:
   ```bash
   docker stats membresia-api --no-stream
   ```
5. Si la API está detenida o colgada, reiniciarla (según `docker ps -a --filter name=membresia-api`):
   ```bash
   docker restart membresia-api
   ```
6. Si el problema persiste, inspeccionar los logs en Grafana (`http://localhost:3000` o mediante el dashboard `nginx-logs` conectado a Loki a través de `alloy`).
````

Anexo 5. Ejemplo de documento autogenerado

Documento generado por `gemini-flash-lite-latest` para la falla F6 cuando el RAG la clasificó como nueva (2,2 segundos). Es el texto que queda guardado en la base de conocimiento:

````
# Incidente: NginxSuspiciousIPActivity - Tráfico excesivo desde IP sospechosa

## Descripción
Se disparó la alerta `NginxSuspiciousIPActivity` (severidad: warning) debido a que la IP `172.18.0.25` generó 1840 peticiones por minuto, superando ampliamente el umbral permitido de 200 req/min. Esto indica un posible comportamiento de scraping automatizado o un ataque de fuerza bruta contra el servicio expuesto en el reverse proxy.

## Causa raíz probable
Un cliente externo o un contenedor dentro de la red Docker está realizando web scraping agresivo, escaneo de directorios o un ataque de denegación de servicio a nivel de aplicación (L7) contra `nginx-proxy`, impactando potencialmente los recursos del contenedor `membresia-api`.

## Pasos de resolución
1. Identificar el origen y los patrones de tráfico accediendo a los logs del contenedor de Nginx mediante el comando:
   `docker logs nginx-proxy --tail 500`
2. Verificar el estado actual de los contenedores para asegurar que el servicio sigue operativo con:
   `docker ps -a`
3. Si el tráfico malicioso persiste y degrada el rendimiento, reiniciar temporalmente el reverse proxy para cortar las conexiones activas ejecutando:
   `docker restart nginx-proxy`
4. Implementar reglas de bloqueo o limitación de tasa (rate limiting) en la configuración de Nginx si la IP continúa afectando la disponibilidad del sistema.

## Prevención
- Configurar límites de tasa (rate limiting) a nivel de Nginx (`limit_req_zone`) para bloquear automáticamente IPs que excedan las 200 req/min.
````

```
- Integrar herramientas de Web Application Firewall (WAF) o reglas en `nginx-proxy` para
detectar y banear patrones de user-agents maliciosos o herramientas de scraping conocidas.
- Monitorear las métricas de `nginx-prometheus-exporter` en Grafana para detectar
anomalías de tráfico de forma temprana.
```

Anexo 6. Rúbrica de evaluación de las propuestas

Tabla 29: Rúbrica de calidad

Criterio	1	3	5
Diagnóstico	Causa equivocada.	Causa genérica ("revisar el servicio").	Identifica la causa de referencia.
Accionabilidad	Sin pasos ejecutables.	Pasos genéricos.	Comandos válidos para este stack, con los nombres reales de los contenedores.
Completitud	Solo describe.	Diagnóstico o arreglo.	Diagnóstico, arreglo y verificación.
Claridad	Confuso o cortado.	Entendible.	Conciso y ordenado.
Alucinaciones	Cantidad de afirmaciones falsas: servicios, archivos o comandos inexistentes.		

Fuente: elaboración propia.

Anexo 7. Material de evaluación

Los scripts de evaluación están en la carpeta `eval/` del repositorio (Tabla 30). El informe completo de la evaluación es `docs/EVALUACION_OE3.md`, los resultados de k6 están en `k6/RESULTS.md` y el detalle de la suite E2E en `docs/FIXES_DASHBOARD.md`.

Tabla 30: Scripts de evaluación

Archivo	Función
`scenarios.json`	Catálogo de fallas F1 a F7 con su causa de referencia, arreglo y clasificación esperada.
`faults.ps1`	Inyecta y restaura una falla; en modo ciego elige una al azar sin mostrarla.
`timer.ps1`	Registra el inicio y el fin de cada alerta consultando Alertmanager.
`mark.ps1`	Registra los hitos manuales de una corrida (inicio, diagnóstico y arreglo).
`summarize-mttr.mjs`	Cruza los registros y calcula MTTR, diagnóstico y tiempo hasta la propuesta.
`run-llm-eval.mjs`	Ejecuta el RAG y cada proveedor con los prompts reales del workflow y genera los archivos para puntuar.
`run-auto.mjs`	Corridas automáticas: inyecta, espera la propuesta, verifica el arreglo, lo aplica y mide.

Fuente: elaboración propia.